

How Does Caddy Fit into the Equation?

Caddy works just like other popular web servers, such as NGINX and Apache. It's best positioned on the edge of your network, between web clients and your web services, as shown in Figure 10-1.

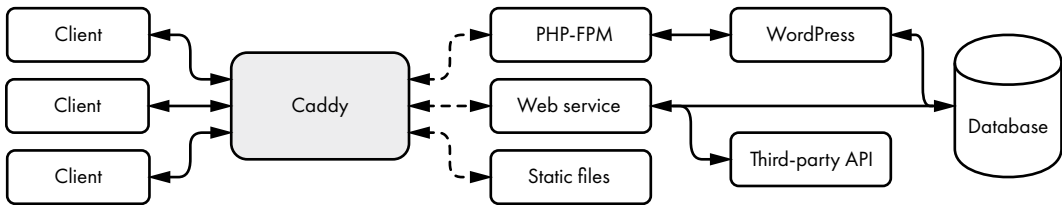


Figure 10-1: Caddy reverse-proxying client requests to web services

Caddy can serve static files and forward requests between clients and backend services, a process known as *reverse proxying*. In this example, you can see Caddy serving a WordPress blog through PHP's FastCGI Process Manager (PHP-FPM), static files, and a Go-based web service. We'll replicate a similar setup later in the chapter, sans WordPress blog.

Caddy helps abstract web services from clients in much the same way we use abstraction in our code. If you use Caddy's automatic TLS, static file server, data compression, access control, and logging features, you won't have to add that functionality to each web service. In addition, using Caddy has the benefit of allowing you to abstract your network topography from clients. As the services increase in popularity and the capacity on web services starts to negatively affect clients, you can add web services to Caddy and instruct Caddy to balance the load among them all, without interruption to your clients.

Retrieving Caddy

In this chapter, we'll use version 2 of Caddy. You have a few options for installation, described in this section.

Downloading Caddy

You can install Caddy by using a static binary, built by the Caddy team. This binary is available through the download link at <https://caddyserver.com/>.

Caddy is also available as a Docker image; a DigitalOcean droplet; an Advanced Package Tool (APT) source for Debian derivatives; and in the Fedora Copr build system for use in Fedora, CentOS, or Red Hat Enterprise Linux. You can find details in the Install documentation at <https://caddyserver.com/docs/download>.

Building Caddy from Source Code

If you do not find a suitable static binary for your operating system and architecture, or if you wish to customize Caddy, you can also compile Caddy from source code.

Caddy relies heavily on Go's support for modules. Therefore, you need to use at least Go 1.14 before running the following commands:

```
$ git clone "https://github.com/caddyserver/caddy.git"
Cloning into 'caddy'...
$ cd caddy/cmd/caddy
$ go build
```

Clone the Caddy Git repository and change to the *caddy/cmd/caddy* subdirectory, where you'll find the main package. Run *go build* to create a binary named *caddy* in the current directory for your operating system and architecture. To simplify commands, the rest of this chapter assumes that the *caddy* binary is in your PATH.

While you're in this subdirectory, make note of the *main.go* file. You'll revisit it later in this chapter when you learn how to customize Caddy by adding modules.

Running and Configuring Caddy

For configuration purposes, Caddy exposes an administration endpoint on TCP port 2019, over which you can interact with Caddy's configuration in real time. You can configure Caddy by posting JSON to this endpoint, and you can read the configuration with a GET request. Caddy's full JSON API documentation is available at <https://caddyserver.com/docs/json/>.

Before you can configure Caddy, you need to start it. Running this command starts Caddy as a background process:

```
$ caddy start
2006/01/02 15:04:05.000 INFO    admin    endpoint started
{"address": "tcp/localhost:2019", "enforce_origin": false,
"origins": ["localhost:2019", "[::1]:2019", "127.0.0.1:2019"]}
2006/01/02 15:04:05.000 INFO    serving initial configuration
Successfully started Caddy (pid=24587) - Caddy is running in the background
```

You'll see log entries showing that the admin endpoint started and Caddy is using the initial configuration. You'll also see log entries printed to standard output as you interact with the admin endpoint.

Caddy's configuration is empty by default. Let's send meaningful configuration data to Caddy. Listing 10-1 uses the *curl* command to post JSON to the load resource on Caddy's admin endpoint.

```
$ curl localhost:2019/load \
❶ -X POST -H "Content-Type: application/json" \
  -d '
```

```
{
  "apps": {
    "http": {
      "servers": {
        "hello": {
          "listen": ["localhost:2020"],
          ❷ "routes": [{
            "handle": [{
              ❸ "handler": "static_response",
                "body": "Hello, world!"
            }]
          }]
        }
      }
    }
  }
}
```

Listing 10-1: Posting configuration to Caddy's admin endpoint

You send a POST request containing JSON in the request body ❶ to the load resource of the Caddy instance listening on port 2019. The top-level `apps` namespace lists the applications Caddy will load at runtime. In this case, you're telling Caddy to load the `http` application. The `http` application configuration consists of one or more servers. This example sets up a single server named `hello` listening on `localhost` port 2020. Feel free to name your server whatever you'd like.

Since the `listen` value is an array of addresses, you can configure this server to listen to more than one socket address. Caddy passes these address values to `net.Listen`, just as you did in Chapter 3. You also have the option of specifying a port range, such as `localhost:2020-2025`. Caddy will recognize that you used a range and properly extrapolate the range into separate socket addresses. Caddy allows you to restrict listeners to specific network types by prefixing the socket address. For example, `udp/localhost:2020` tells the server to bind to UDP port 2020 on `localhost`. The forward slash is not part of the address but rather a separator. If you want the server to bind to a Unix socket `/tmp/caddy.sock`, specify the address `unix//tmp/caddy.sock`.

The `hello` server's `routes` value ❷ is an array of routes, like the multiplexer from the preceding chapter, which dictates how the server will handle incoming requests. If a route matches the request, Caddy passes the request onto each handler in the `handle` array. Since `handle` is an array, you can specify more than one handler per route. Caddy will pass the request to each successive handler in the same way you chained middleware together in the preceding chapter. In this example, you specify a single route to match all requests and add a single handler to this route. You're using the built-in `static_response` handler ❸, which will write the value of the body (`Hello, world!` in this example) in the response body.

Provided there are no errors in the configuration, Caddy will at once start using the new configuration. Let's confirm Caddy is now listening on both the administrative port 2019 and your hello server port 2020:

```
$ lsof -Pi :2019-2025
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
caddy	24587	user	3u	IPv4	811511	0t0	TCP	localhost:2019 (LISTEN)
caddy	24587	user	9u	IPv4	915742	0t0	TCP	localhost:2020 (LISTEN)

Looks good. This command won't work on Windows. Instead, you can see similar output by running the `netstat -b` command in an Administrator command prompt. Now, you can ask Caddy for its configuration by sending a GET request:

```
$ curl localhost:2019/config/
```

```
{"apps":{"http":{"servers":{"hello":{"listen":["localhost:2020"]},  
"routes":[{"handle":[{"body":"Hello, world!","handler":"static_  
response"}]}]}}}}}
```

Caddy returns its JSON-formatted configuration in the response body. Note that you need to write the trailing slash on the `/config/` resource, because `/config/` is the resource prefix under which Caddy exposes its configuration. You are asking Caddy for all resources found under the `/config/` prefix. If you accidentally omit the trailing slash, Caddy thinks you're asking for an absolute resource named `/config`, which doesn't exist in Caddy's admin API on port 2019.

Caddy supports *configuration traversal*. Configuration traversal lets you request a subset of the configuration by treating each JSON key in the configuration data as a resource address. For example, you can request the `listen` value for the `hello` server from our example configuration by issuing a GET request, like this:

```
$ curl localhost:2019/config/apps/http/servers/hello/listen  
["localhost:2020"]
```

Caddy returns a JSON array containing `localhost:2020`, just as you'd expect. Let's send a GET request to this socket address:

```
$ curl localhost:2020  
Hello, world!
```

You see the `Hello, world!` string returned from the `static_response` handler.

Modifying Caddy's Configuration in Real Time

You can use the other HTTP verbs you learned in Chapter 8 to modify your server's configuration. Any changes you make will take immediate effect, so long as Caddy can parse the JSON you send. If Caddy fails to parse the JSON, or if a fundamental error exists in the new configuration, Caddy will log an error with an explanation of what went wrong and continue to use its existing configuration.

Let's say you want to make your hello server listen on port 2021 as well. You can append another listen value by using a POST request and immediately check that the change took effect:

```
$ curl localhost:2019/config/apps/http/servers/hello/listen \
-X POST -H "Content-Type: application/json" -d '{"localhost:2021"}'
$ lsof -Pi :2019-2025
```

	COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
	caddy	24587	user	3u	IPv4	811511	0t0	TCP	localhost:2019 (LISTEN)
	caddy	24587	user	9u	IPv4	915742	0t0	TCP	localhost:2020 (LISTEN)
❶	caddy	24587	user	11u	IPv4	1148212	0t0	TCP	localhost:2021 (LISTEN)

You can see that Caddy is now listening on port 2021 ❶ in addition to ports 2019 and 2020.

Suppose you want to replace the listening addresses and use a range instead. For that, you can send a PATCH request with the new listen array value you want Caddy to use:

```
$ curl localhost:2019/config/apps/http/servers/hello/listen \
-X PATCH -H "Content-Type: application/json" -d '["localhost:2020-2025"]'
$ lsof -Pi :2019-2025
```

	COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
	caddy	24587	user	3u	IPv4	811511	0t0	TCP	localhost:2019 (LISTEN)
❶	caddy	24587	user	9u	IPv4	915742	0t0	TCP	localhost:2020 (LISTEN)
	caddy	24587	user	10u	IPv4	1149557	0t0	TCP	localhost:2021 (LISTEN)
	caddy	24587	user	11u	IPv4	1166333	0t0	TCP	localhost:2022 (LISTEN)
	caddy	24587	user	12u	IPv4	1169409	0t0	TCP	localhost:2023 (LISTEN)
	caddy	24587	user	13u	IPv4	1169413	0t0	TCP	localhost:2024 (LISTEN)
❷	caddy	24587	user	14u	IPv4	1169417	0t0	TCP	localhost:2025 (LISTEN)

In addition to the admin port 2019, Caddy is now listening on ports 2020 ❶ through 2025 ❷.

Although you may not find yourself changing Caddy's configuration on the fly very often, it's a handy feature for development, because it lets you quickly spin up a new server to add functionality. Let's add a new server to Caddy while it's running. You'll name this new server test and configure it to listen on port 2030. Listing 10-2 adds the new test server to Caddy in real time.

```
$ curl localhost:2019/config/apps/http/servers/test \
-X POST -H "Content-Type: application/json" \
-d '{
  "listen": ["localhost:2030"],
  "routes": [{
    "handle": [{
      "handler": "static_response",
      "body": "Welcome to my temporary test server."
    }]
  }]
}'
```

Listing 10-2: Adding a new server to Caddy in real time

The name of the new server, `test`, is part of the resource you `POST` to. You can think of `test` as the key and the JSON in the request body as the value, if you defined this server in the original configuration from Listing 10-1. At this point, Caddy has two servers: `hello` listening on ports 2020 to 2025 and `test` listening on port 2030. To confirm Caddy is serving `test`, you can check the new endpoint on port 2030:

```
$ curl localhost:2030
Welcome to my temporary test server.
```

The `static_response` handler properly responds with the expected message. If you want to remove the test server, it's as simple as issuing a `DELETE` request:

```
$ curl localhost:2019/config/apps/http/servers/test -X DELETE
```

Here again, you specify the test server in the resource. Caddy is no longer listening on localhost port 2030, and the test server no longer exists. You were able to stand up a new server to handle entirely different requests without interrupting the functionality of your `hello` server. Changing the configuration in real time opens possibilities. Do you want a server or route to be accessible only certain times of the day? No problem. Do you want to temporarily redirect traffic without having to bounce your entire web server, interrupting existing web traffic? Sure, go ahead.

Storing the Configuration in a File

We typically provide Caddy with its configuration as part of the startup process. Write the JSON configuration from Listing 10-1 to a file named *caddy.json*. Then start Caddy by using the following command:

```
$ caddy start --config caddy.json
Successfully started Caddy (pid=46112) - Caddy is running in the background
$ curl localhost:2019/config/
{"apps":{"http":{"servers":{"hello":{"listen":["localhost:2020"],
"routes":[{"handle":[{"body":"Hello, world!","handler":"static_
response"}]}]}}}}}}
```

Caddy starts in the background, as in Listing 10-1—but this time, it populates its configuration from the *caddy.json* file during initialization.

Extending Caddy with Modules and Adapters

Caddy uses a modular architecture to organize its functionality. This modular approach allows you to extend Caddy's capabilities by writing your own modules and configuration adapters. In this section, we'll walk through the process of writing a configuration adapter that will allow you to store your Caddy configuration in a Tom's Obvious, Minimal Language (TOML) file. We'll also replicate the `restrict_prefix` middleware from the preceding chapter in a proper Caddy module.

Writing a Configuration Adapter

Although JSON is a perfectly good format for configuration files, it isn't as well suited for human consumption as other formats. JSON lacks support for comments and multiline strings, two characteristics that make configuration files easier for people to read. Caddy supports the use of *configuration adapters* that adapt one format, such as TOML, to Caddy's native JSON format. TOML is a configuration file format that is easy for humans to read. It supports both comments and multiline strings. You can find more details at <https://github.com/toml-lang/toml/tree/v0.5.0/>.

Caddy version 1 supported a custom configuration file format named *Caddyfile*, which was also the name of the configuration file by convention. If you want to use Caddyfile with Caddy v2, you must rely on a configuration adapter so Caddy can ingest it. Caddy is smart enough to know it needs to use the *caddyfile* adapter when you specify a filename that starts with *Caddyfile*. But to specify an adapter from the command line, you explicitly tell Caddy which adapter to use:

```
$ caddy start --config Caddyfile --adapter caddyfile
```

The adapter flag tells Caddy which adapter it should use. Caddy will invoke the adapter to adapt the configuration file to JSON and then parse the JSON returned by the adapter as if you had presented the configuration in JSON format in the first place.

But Caddy doesn't ship with an official configuration adapter for TOML, so let's take a crack at writing one. You need to first create a Go module for your TOML configuration adapter:

```
$ mkdir caddy-toml-adapter
$ cd caddy-toml-adapter
❶ $ go mod init github.com/awoodbeck/caddy-toml-adapter
go: creating new go.mod: module github.com/awoodbeck/caddy-toml-adapter
```

You should use a fully qualified module name ❶ different from the one used here. I created this module on GitHub under my *awoodbeck* account. The fully qualified name for your module will differ depending on where, and under what account, it's hosted.

Now that you've created a module, you can write the code. Create a file in the current directory named *toml.go* and add the code in Listing 10-3.

```
package tomladapter

import (
    "encoding/json"

    "github.com/caddyserver/caddy/v2/caddyconfig"
    "github.com/pelletier/go-toml"
)

func init() {
    caddyconfig.RegisterAdapter(❶ "toml", ❷ Adapter{})
```

```

}

// Adapter converts a TOML Caddy configuration to JSON.
type Adapter struct{}

// Adapt the TOML body to JSON.
func (a Adapter) Adapt(body []byte, _ map[string]interface{}) (
    []byte, []caddyconfig.Warning, error) {
    tree, err := ❸toml.LoadBytes(body)
    if err != nil {
        return nil, nil, err
    }

    b, err := json.Marshal(❹tree.ToMap())

    return b, nil, err
}

```

Listing 10-3: Creating a TOML configuration adapter and registering it with Caddy

You use Thomas Pelletier’s *go-toml* library to parse the configuration file contents ❸. This saves a considerable amount of code. You then convert the parsed TOML into a map ❹ and marshal the map to JSON.

The last bit of accounting is to register your configuration adapter with Caddy. For this, you include a call to `caddyconfig.RegisterAdapter` in the `init` function and pass it the adapter’s type ❶ and an `Adapter` object ❷ implementing the `caddyconfig.Adapter` interface. When you import this module from Caddy’s *main.go* file, the configuration adapter registers itself with Caddy, adding support for parsing the TOML configuration file. You’ll look at a concrete example of importing this module from Caddy in “Injecting Your Module into Caddy” on page 231.

Now that you’ve created the *toml.go* file, tidy up the module:

```

$ go mod tidy
go: finding module for package github.com/caddyserver/caddy/v2/caddyconfig
go: found github.com/caddyserver/caddy/v2/caddyconfig in
github.com/caddyserver/caddy/v2 v2.0.0

```

This command adds the Caddy dependency to the *go.mod* file. All that’s left to do is to publish your module to GitHub, as in this example, or another suitable version-control system supported by `go get`.

Writing a Restrict Prefix Middleware Module

Chapter 9 introduced the concept of middleware, a design pattern that allows your code to manipulate a request and a response and to perform ancillary tasks when the server receives a request, such as logging request details. Let’s explore how to use middleware with Caddy.

In Go, middleware is a function that accepts an `http.Handler` and returns an `http.Handler`:

```

func(http.Handler) http.Handler

```

An `http.Handler` describes an object with a `ServeHTTP` method that accepts an `http.ResponseWriter` and an `http.Request`:

```
type Handler interface {  
    ServeHTTP(http.ResponseWriter, *http.Request)  
}
```

The handler reads from the request and writes to the response. Assuming `myHandler` is an object that implements the `http.Handler` interface, and `middleware1`, `middleware2`, and `middleware3` all accept an `http.Handler` and return an `http.Handler`, you can apply the middleware functions to `myHandler` in Listing 10-4.

```
h := middleware1(middleware2(middleware3(myHandler)))
```

Listing 10-4: Multiple middleware functions wrapping a handler

You can replace any of the middleware functions with the `RestrictPrefix` middleware you wrote in the preceding chapter, since it's a function that accepts an `http.Handler` and returns an `http.Handler`.

Unfortunately for us, Caddy's middleware does not use this design pattern, so it cannot use `RestrictPrefix`. Caddy includes interfaces for both handlers and middleware, unlike `net/http`, which describes only handlers. Caddy's equivalent of the `http.Handler` interface is `caddyhttp.Handler`:

```
type Handler interface {  
    ServeHTTP(http.ResponseWriter, *http.Request) error  
}
```

The only difference between `caddyhttp.Handler` and `http.Handler` is that the former's `ServeHTTP` method returns an error interface.

Caddy middleware is a special type of handler that implements the `caddyhttp.MiddlewareHandler` interface:

```
type MiddlewareHandler interface {  
    ServeHTTP(http.ResponseWriter, *http.Request, Handler) error  
}
```

Like `caddyhttp.Handler`, Caddy's middleware accepts both an `http.ResponseWriter` and an `http.Request`, and it returns an error interface. But it accepts an additional argument: the `caddyhttp.Handler`, downstream from the middleware in the same way that `myHandler` is downstream from `middleware3` in Listing 10-4. Instead of accepting an `http.Handler` and returning an `http.Handler`, Caddy expects its middleware to act as handlers, with access to the `caddyhttp.Handler` that should receive the request and response after the middleware is done with them.

Let's create a new Caddy module that replicates the functionality of your `RestrictPrefix` middleware:

```
$ mkdir caddy-restrict-prefix  
$ cd caddy-restrict-prefix
```

```
$ go mod init github.com/awoodbeck/caddy-restrict-prefix
go: creating new go.mod: module github.com/awoodbeck/caddy-restrict-prefix
```

As before, your fully qualified module name will differ from mine. Create a new file named *restrict_prefix.go* and add the code from Listing 10-5 to the file.

```
package restrictprefix

import (
    "fmt"
    "net/http"
    "strings"

    "github.com/caddyserver/caddy/v2"
    "github.com/caddyserver/caddy/v2/modules/caddyhttp"
    "go.uber.org/zap"
)

func init() {
    ❶ caddy.RegisterModule(RestrictPrefix{})
}

// RestrictPrefix is middleware that restricts requests where any portion
// of the URI matches a given prefix.
type RestrictPrefix struct {
    ❷ Prefix string `json:"prefix,omitempty"`
    ❸ logger *zap.Logger
}

// CaddyModule returns the Caddy module information.
func (RestrictPrefix) ❹ CaddyModule() caddy.ModuleInfo {
    return caddy.ModuleInfo{
        ❺ ID: "http.handlers.restrict_prefix",
        ❻ New: func() caddy.Module { return new(RestrictPrefix) },
    }
}
```

Listing 10-5: Defining and registering a new Caddy module

The `RestrictPrefix` middleware implementation from the preceding chapter expected the prefix of a URL path as a string. Here, you're storing the prefix in the `RestrictPrefix` struct ❷ and assigning it a struct tag to use the `json.Unmarshal` behavior of matching incoming keys to struct tags. The struct tag tells `json.Unmarshal` which JSON key corresponds to this field. In this example, you're telling `json.Unmarshal` that it should take the value associated with the `prefix` key in the JSON configuration and assign it to the struct's `Prefix` field. The `RestrictPrefix` struct also has a `logger` field ❸ so you can log events, as necessary.

Your module needs to register itself with Caddy upon initialization ❶. The `caddy.RegisterModule` function accepts any object that implements the `caddy.Module` interface. For that, you add the `CaddyModule` method ❹ to return

information to Caddy about your module. Caddy requires an ID **5** for each module. Since you're creating an HTTP middleware handler, you'll use the ID `http.handler.restrict_prefix`, where `restrict_prefix` is the unique name of your module. Caddy also expects a function **6** that can create a new instance of your module.

Now that you can register your module with Caddy, let's add more functionality so you can retrieve the logger from Caddy and validate your module's settings. Listing 10-6 picks up where we left off.

--snip--

```
// Provision a Zap logger to RestrictPrefix.
func (p *RestrictPrefix) ❶Provision(ctx caddy.Context) error {
    p.logger = ❷ctx.Logger(p)
    return nil
}

// Validate the prefix from the module's configuration, setting the
// default prefix "." if necessary.
func (p *RestrictPrefix) ❸Validate() error {
    if p.Prefix == "" {
        p.Prefix = "."
    }
    return nil
}
```

Listing 10-6: Implementing various Caddy interfaces

You add the `Provision` method **1** to your struct. Caddy will recognize that your module implements the `caddy.Provisioner` interface and call this method. You can then retrieve the logger from the given `caddy.Context` **2**. Likewise, Caddy will call your module's `Validate` method **3** since it implements the `caddy.Validator` interface. You can use this method to make sure all required settings have been unmarshaled from the configuration into your module. If anything goes wrong, you can return an error and Caddy will complain on your behalf. In this example, you're using this method to set the default prefix if one was not provided in the configuration.

You're almost done. The last piece of the puzzle is the middleware implementation itself. Listing 10-7 rounds out your module's implementation by adding support for the `caddyhttp.MiddlewareHandler` interface.

--snip--

```
// ServeHTTP implements the caddyhttp.MiddlewareHandler interface.
func (p RestrictPrefix) ServeHTTP(w http.ResponseWriter, r *http.Request,
    next caddyhttp.Handler) error {
    ❶ for _, part := range strings.Split(r.URL.Path, "/") {
        if strings.HasPrefix(part, p.Prefix) {
            ❷http.Error(w, "Not Found", http.StatusNotFound)
            if p.logger != nil {
                ❸p.logger.Debug(fmt.Sprintf(
                    "restricted prefix: %q in %s", part, r.URL.Path))
            }
        }
    }
    return next.ServeHTTP(w, r)
}
```

```

        }
        return nil
    }
}
return ❹next.ServeHTTP(w, r)
}

var (
    ❺ _ caddy.Provisioner           = (*RestrictPrefix)(nil)
    _ caddy.Validator              = (*RestrictPrefix)(nil)
    _ caddyhttp.MiddlewareHandler = (*RestrictPrefix)(nil)
)

```

Listing 10-7: Implementing the `MiddlewareHandler` interface

The logic is almost identical to the middleware from the preceding chapter. You loop through the URL path components, checking each one for the prefix ❶. If you find a match, you respond with a 404 Not Found status ❷ and log the occurrence for debugging purposes ❸. If everything checks out, you pass control onto the next handler in the chain ❹.

It's a good practice to guard against interface changes by explicitly making sure your module implements the expected interfaces ❺. If one of these interfaces happens to change in the future (for example, if you add a new method), these interface guards will cause compilation to fail, giving you an early warning that you need to adapt your code.

The final steps are to tidy up your module's dependencies and publish it:

```

$ go mod tidy
go: finding module for package github.com/caddyserver/caddy/v2
go: finding module for package github.com/caddyserver/caddy/v2/modules/
caddyhttp
go: finding module for package go.uber.org/zap
go: found github.com/caddyserver/caddy/v2 in github.com/caddyserver/caddy/v2
v2.0.0
go: found go.uber.org/zap in go.uber.org/zap v1.15.0
go: downloading github.com/golang/mock v1.4.1
go: downloading github.com/onsi/gomega v1.8.1
go: downloading github.com/smallstep/assert v0.0.0-20200103212524-b99dc1097b15
go: downloading github.com/onsi/ginkgo v1.11.0
go: downloading github.com/imdario/mergo v0.3.7
go: downloading github.com/chzyer/test v0.0.0-20180213035817-a1ea475d72b1
go: downloading github.com/golang/glog v0.0.0-20160126235308-23def4e6c14b
go: downloading github.com/alangpierce/go-forceexport v0.0.0-20160317203124-
8f1d6941cd75
go: downloading github.com/chzyer/logex v1.1.10
go: downloading github.com/hpcloud/tail v1.0.0
go: downloading gopkg.in/tomb.v1 v1.0.0-20141024135613-dd632973f1e7
go: downloading gopkg.in/fsnotify.v1 v1.4.7

```

Publish your module to GitHub or a similar version-control system supported by go get.

Injecting Your Module into Caddy

The module and adapter you wrote are both self-registering. All you need to do to include their functionality in Caddy is to import them at build time. To do that, you need to compile Caddy from source. Start by making a directory for your build:

```
$ mkdir caddy
$ cd caddy
```

Building Caddy from source code requires a small amount of boilerplate code, to which you'll include your modules. Your modules register themselves with Caddy as a side effect of the import. Create a new file named *main.go* and add the code from Listing 10-8 into it.

```
package main

import (
    ❶ cmd "github.com/caddyserver/caddy/v2/cmd"
    ❷ _ "github.com/caddyserver/caddy/v2/modules/standard"

    // Injecting custom modules into Caddy
    ❸ _ "github.com/awoodbeck/caddy-restrict-prefix"
    ❹ _ "github.com/awoodbeck/caddy-toml-adapter"
)

func main() {
    cmd.Main()
}
```

Listing 10-8: Injecting custom modules into Caddy

First, you import the Caddy command module ❶ into your build. This has the `Main` function that starts the caddy server. Then, you import the standard modules ❷ that you'll find in Caddy's binary distribution. Finally, you include your restrict prefix module ❸ and your TOML configuration adapter ❹.

All that's left to do now is initialize the caddy module and build it:

```
$ go mod init caddy
$ go build
```

At this point, you should have a binary named *caddy* in the current directory. You can verify that it has your custom imports by looking for them in the caddy binary's list of modules. The following command is specific to Linux and macOS:

```
$ ./caddy list-modules | grep "toml\|restrict_prefix"
caddy.adapters.toml
http.handlers.restrict_prefix
```

For my Windows friends, run this command instead:

```
> caddy list-modules | findstr "toml restrict_prefix"
caddy.adapters.toml
http.handlers.restrict_prefix
```

The caddy binary you built can read its configuration from TOML files and deny clients access to resources whose path includes a given prefix.

Reverse-Proxying Requests to a Backend Web Service

You now have all the building blocks to create something meaningful in Caddy. Let's put everything you've learned together by configuring Caddy to reverse-proxy requests to a backend web service and serve up static files on behalf of the backend web service. You'll create two endpoints in Caddy. The first endpoint will serve up only static content from Caddy's file server, showing Caddy's static file-serving abilities. The second endpoint will reverse-proxy requests to a backend web service. This backend service will send the client HTML that will prompt the client to retrieve static files from Caddy, which will show how your web services can lean on Caddy to serve static content on their behalf.

Before you start building, you need to set up the proper directory structure. If you're following along, you're currently in the *caddy* directory, which has a caddy binary built from the code in Listing 10-8. Create two subdirectories, *files* and *backend*:

```
$ mkdir files backend
```

You can retrieve the contents of the *files* subdirectory from <https://github.com/awoodbeck/gnp/tree/master/ch10/files/>. The *backend* subdirectory will store a simple backend service created in the next section.

Creating a Simple Backend Web Service

You need a backend web service for Caddy to reverse-proxy requests to, as illustrated in Figure 10-1. This service will respond to all requests with an HTML document that includes the static files Caddy serves on the service's behalf.

Listing 10-9 is the initial code for the backend web service.

```
package main

import (
    "flag"
    "fmt"
    "log"
    "net/http"
    "os"
    "os/signal"
    "time"
)
```

```

var addr = flag.String("listen", ❶"localhost:8080", "listen address")

func main() {
    flag.Parse()

    c := make(chan os.Signal, 1)
    signal.Notify(c, os.Interrupt)

    err := ❷run(*addr, c)
    if err != nil {
        log.Fatal(err)
    }

    log.Println("Server stopped")
}

```

Listing 10-9: Creating a backend service (backend/main.go)

This bit of code should be familiar since it's a simplified version of what you wrote in the preceding chapter. You're setting up a web service that listens on port 8080 of localhost ❶. Caddy will direct requests to this socket address. Listing 10-10 implements the run function ❷.

--snip--

```

func run(addr string, c chan os.Signal) error {
    mux := http.NewServeMux()
    mux.Handle("/",
        http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            clientAddr := r.Header.Get(❶"X-Forwarded-For")
            log.Printf("%s -> %s -> %s", clientAddr, r.RemoteAddr, r.URL)
            _, _ = w.Write(❷index)
        })),
    )

    srv := &http.Server{
        Addr:      addr,
        Handler:    mux,
        IdleTimeout: time.Minute,
        ReadHeaderTimeout: 30 * time.Second,
    }

    go func() {
        for {
            if <-c == os.Interrupt {
                _ = srv.Close()
                return
            }
        }
    }()

    fmt.Printf("Listening on %s ...\n", srv.Addr)
    err := srv.ListenAndServe()
}

```

```

    if err == http.ErrServerClosed {
        err = nil
    }

    return err
}

```

Listing 10-10: The main logic of the backend service (backend/main.go)

The web service receives all requests from Caddy, no matter which client originated the request. Likewise, it sends all responses back to Caddy, which then routes the response to the right client. Conveniently, Caddy adds an X-Forwarded-For header ❶ to each request with the originating client's IP address. Although you don't do anything other than log this information, your backend service could use this IP address to differentiate between client requests. The service could deny requests based on client IP address, for example.

The handler writes a slice of bytes ❷ to the response that has HTML defined in Listing 10-11.

--snip--

```

var index = []byte(`<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Caddy Backend Test</title>
    <link href=❶"/style.css" rel="stylesheet">
</head>
<body>
    <p><img src=❷"/hiking.svg" alt="hiking gopher"></p>
</body>
</html>`)

```

Listing 10-11: The index HTML served by the backend service (backend/main.go)

The `/style.css` ❶ and `/hiking.svg` ❷ resources do not include a full URL (such as `http://localhost:2020/style.css`) because the backend web service does not know anything about Caddy or how clients access Caddy. When you exclude the scheme, hostname, and port number in the resource address, the client's web browser should encounter `/style.css` in the HTML and prepend the scheme, hostname, and port number it used for the initial request before sending the request to Caddy. For that all to work, you need to configure Caddy in the next section to send some requests to the backend web service and serve static files for the rest of the requests.

Setting Up Caddy's Configuration

As mentioned earlier in the chapter, Caddy uses JSON as its native configuration format. You could certainly write your configuration in JSON, but you've already written a perfectly good configuration adapter that allows you to use TOML, so you'll implement that instead.

You want to configure Caddy to reverse-proxy requests to your backend web service and serve static files from the *files* subdirectory. You'll need two routes: one to the backend web service and one for static files. Let's start by defining your server configuration in a file named *caddy.toml* (Listing 10-12).

```
❶ [apps.http.servers.test_server]
listen = [
    'localhost:2020',
]
```

Listing 10-12: Caddy test server configuration (caddy.toml)

Your TOML adapter directly converts TOML to JSON. Therefore, you need to make sure you're using the same namespaces Caddy expects. The namespace for your server is `apps.http.servers.test_server` **❶**. (For simplicity, you'll refer to this namespace simply as `test_server` from here on out.) It listens for incoming connections on port 2020 of localhost.

Adding a Reverse-Proxy to Your Service

Caddy includes a powerful reverse-proxy handler that makes quick work of sending incoming requests to your backend web service. Just as in the server implementation in the preceding chapter, Caddy matches an incoming request to a route and then passes the request onto the associated handler.

Listing 10-13 adds a route and a reverse-proxy handler to the *caddy.toml* file.

--snip--

```
❶ [[apps.http.servers.test_server.routes]]
❷ [[apps.http.servers.test_server.routes.match]]
path = [
    '/backend',
    ❸ '/backend/*',
]
❹ [[apps.http.servers.test_server.routes.handle]]
handler = 'reverse_proxy'
❺ [[apps.http.servers.test_server.routes.handle.upstreams]]
dial = ❻ 'localhost:8080'
```

Listing 10-13: Adding a reverse proxy to the backend service (caddy.toml)

The `test_server` configuration includes a `routes` array **❶**, and each route in the array has zero or more `matchers` **❷**. A *matcher* is a special module that allows you to specify matching criteria for incoming requests, like the `http.ServeMux.Handle` method's pattern matching discussed in the preceding chapter. Caddy includes matcher modules that allow you to consider each part of a request.

For this route, you add a single matcher that matches any request for the absolute path `/backend` or any path starting with `/backend/` **❸**. The `*` character is a wildcard that tells Caddy you want to match on the `/backend/` prefix. For example, a request for the resource `/backend/this/is/a/test` will also match.

The route may have one or more handlers ❹. Here, you tell Caddy you want to send all matching requests to the reverse-proxy handler. The reverse-proxy handler needs to know where to send the requests. You specify an upstream entry ❺ with its dial property set to the backend server's socket address ❻.

Serving Static Files

You relied on the `http.FileServer` to serve static files for you in the preceding chapter. Caddy exposes similar functionality with its `file_server` handler. Listing 10-14 adds a second route to your *caddy.toml* file for serving static files.

--snip--

```
❶ [[apps.http.servers.test_server.routes]]
❷ [[apps.http.servers.test_server.routes.handle]]
  handler = 'restrict_prefix'
  prefix = '.'
❸ [[apps.http.servers.test_server.routes.handle]]
  handler = 'file_server'
  root = ❹ './files'
  index_names = [
    ❺ 'index.html',
  ]
```

Listing 10-14: Adding a default route to serve static files (caddy.toml)

Unlike the route you added in Listing 10-13, this route ❶ does not include any matchers. As such, Caddy would send every request to this route's handler if the request didn't match previous routes. In other words, this route is your default route, and so its position in the file matters. If you moved this route before the reverse-proxy route, all requests would match it, and no requests would ever make their way to the reverse proxy. Whenever you specify a route with no matches, make sure you put it at the end of your routes array, as you do here.

As with the file server in the preceding chapter, you want to protect against accidentally serving sensitive files prefixed with a period. Therefore, you include your `restrict_prefix` middleware ❷ in the array of handlers before the `file_server` handler ❸. You add more configuration options to serve files found in the *files* subdirectory ❹ and return the *index.html* file ❺ if the request didn't specify a file.

Checking Your Work

Everything is in place. Start Caddy and verify that your configuration works as expected. Since some of the static files are images, I recommend you use a web browser to interact with Caddy while it runs on your computer.

Start Caddy by using the *caddy.toml* file and the *toml* adapter:

```
$ ./caddy start --config caddy.toml --adapter toml
```

On Windows, the command looks like this:

```
> caddy start --config caddy.toml --adapter toml
```

Now, run the backend web service:

```
$ cd backend
$ go run backend.go
Listening on localhost:8080 ...
```

Open your web browser and visit <http://localhost:2020/>. Caddy will send your request to the file server handler, which in turn will respond with the *index.html* file, since you didn't indicate a specific file in the request. Your browser then asks Caddy for the *style.css* and *sage.svg* files to finish rendering the page. If everything succeeds, you should now be looking at a sage gopher.

Now, let's test the reverse proxy to the backend web service. Visit <http://localhost:2020/backend>. This request matches the reverse-proxy route's matcher, so the reverse-proxy handler should handle it, sending the request onto the backend service. The backend web service responds with HTML that instructs your browser to retrieve the *style.css* and *hiking.svg* files from Caddy, where the file server handler happily serves them up. You should now be looking at a hiking gopher rendered using HTML from the backend web service and static files from Caddy.

If you copied the *files* subdirectory from this book's source code repository, it should contain *./files/.secret* and *./files/.dir/secret* files. Your middleware should block access to both files. In other words, both <http://localhost:2020/files/.secret> and <http://localhost:2020/files/.dir/secret> will return a 404 Not Found status if you try to request them.

Adding Automatic HTTPS

Now let's add Caddy's key feature to your web server: automatic HTTPS.

I once used Caddy to stand up a website with full HTTPS support, using certificates trusted by all contemporary web browsers, in a matter of minutes. The server has been rock-solid ever since, happily rotating Let's Encrypt keys every few months with no intervention on my part. This isn't to say I couldn't replicate this functionality in my own Go-based web server; my time was simply best spent building services and leaving the web serving to Caddy. If Caddy lacked any functionality, I could add it as a module.

Caddy automatically enables TLS when it can determine what domain names you've configured it to serve. The *caddy.toml* configuration created in this chapter didn't give Caddy enough information to determine which domain it was serving. Therefore, Caddy didn't enable HTTPS for you. You told Caddy to bind to localhost, but that tells Caddy only what it's listening to, not what domains it's serving.

The most common way to enable automatic HTTPS is by adding a host matcher to one of Caddy's routes. Here's an example matcher:

```
[[apps.http.servers.server.routes.match]]
host = [
    'example.com',
]
```

This host matcher supplies enough information for Caddy to determine that it is serving the *example.com* domain. If Caddy doesn't already have a valid certificate for *example.com* to enable HTTPS, it will go through the process of validating the domain with Let's Encrypt and retrieving a certificate. Caddy will manage your certificate, automatically renewing it, as necessary.

Caddy's file-server subcommand tells Caddy you want it to exclusively serve files over HTTP. The file-server's `--domain` flag is enough information for Caddy to invoke its automatic HTTPS and allow you to serve files over HTTPS as well.

Caddy's reverse-proxy subcommand allows you to put Caddy into a reverse-proxy-only mode, where it will send all incoming requests onto the socket address specified by the `--to` flag. Caddy will retrieve a TLS certificate and enable automatic HTTPS if you specify a hostname with the `--from` flag.

I encourage you to read more about Caddy's automatic HTTPS in production environments at <https://caddyserver.com/docs/automatic-https>.

NOTE

Caddy defaults to using Let's Encrypt as its certificate authority for non-localhost names and IP addresses. But Caddy supports its own internal certificate authority for enabling HTTPS over localhost, which you may want to do for testing purposes. If you specify localhost as your hostname in the earlier settings to enable automatic HTTPS, Caddy will use its internal certificate authority for its TLS certificate. It will also try to install its root certificate in your operating system's root certificate trust store, which is where your operating system keeps track of all root certificates it inherently trusts.

What You've Learned

Caddy is a contemporary web server written in Go that offers security, performance, and extensibility through modules and configuration adapters. Caddy can automatically use HTTPS through its integration with Let's Encrypt, a nonprofit certificate authority that supplies free digital certificates. Together, Caddy and Let's Encrypt allow you to stand up a web server with seamless HTTPS support.

Caddy uses JSON as its native configuration format, and it exposes an API on localhost port 2019 that allows you to post JSON to change its configuration. The configuration changes take immediate effect. But since JSON isn't an ideal configuration format, Caddy makes use of configuration adapters. Configuration adapters translate configuration files from more configuration-friendly formats, like TOML, to JSON. If you don't want to

use JSON for your Caddy configuration or if you don't find a configuration adapter that meets your needs, you can also write your own, as you did in this chapter.

You can also extend Caddy's functionality with the use of modules. This chapter shows how to write a middleware module, compile it into Caddy, configure the module, and put it to effective use.

Finally, this chapter showed you how to integrate Caddy into your network architecture. You'll often make Caddy the first server in your network, using it to receive client requests before sending the requests onto their final destinations. In this chapter, you configured an instance of Caddy to reverse-proxy client requests to your backend web service and serve static files on behalf of the backend web service. As a result, you kept your backend web service as simple as possible and saved it from having to manage its static content. Your backend web service can use Caddy for HTTPS support, caching, and file serving.

Now that you have some experience with Caddy, you should be able to determine whether your web services would do better when served by a comprehensive web server solution or a comparatively minimal net/http web server implementation. If you expect to make your web service available to the public, using a proven web server like Caddy at the edge of your application will free up time you can better spend on your backend web service.

11

SECURING COMMUNICATIONS WITH TLS



Five years before whistleblower Edward Snowden showed us how much we took our electronic privacy for granted, author and activist Cory Doctorow wrote, “We should treat personal electronic data with the same care and respect as weapons-grade plutonium—it is dangerous, long-lasting, and once it has leaked, there’s no getting it back.”

Prior to 2013, most people communicated on the internet by using plaintext. Social Security numbers, credit card details, passwords, sensitive emails, and other potentially embarrassing information traveled over the internet, ripe for interception by malicious actors. Most popular websites defaulted to HTTP; Google was one of the only major tech companies supporting HTTPS.

Today, it’s unusual to find a website that doesn’t support HTTPS, particularly now that Let’s Encrypt offers free TLS certificates for your domain. We’re treating information in transit more like weapons-grade plutonium,

helping ensure the privacy and integrity of the information we share. Our network applications should be no different. We should strive to authenticate our communication and use encryption where appropriate, particularly when that information has the potential to leak over insecure networks.

Up to this point, we've used TLS only as an afterthought in our code. This is partly because Go's net/http library makes its use relatively effortless, but it's also because we haven't adequately explored the TLS protocol and the infrastructure that makes it possible. To write secure software, you should carefully plan for security before development starts and then use good security practices as you write code. TLS is a terrific way to improve the security posture of your software by protecting data in transit.

This chapter will introduce you to the basics of TLS from a programmer's perspective. You'll learn about the client-server handshake process and the inherent trust that makes that process work. Then we'll discuss how things can (and do) go wrong even when you use TLS. Finally, we'll look at practical examples of how to incorporate TLS into your applications, including mutual client-server authentication.

A Closer Look at Transport Layer Security

The TLS protocol supplies secure communication between a client and a server. It allows the client to authenticate the server and optionally permits the server to authenticate clients. The client uses TLS to encrypt its communication with the server, preventing third-party interception and manipulation.

TLS uses a handshake process to establish certain criteria for the stateful TLS session. If the client initiated a TLS 1.3 handshake with the server, it would go something like this:

Client Hello *google.com*. I'd like to communicate with you using TLS version 1.3. Here is a list of ciphers I'd like to use to encrypt our messages, in order of my preference. I generated a public- and private-key pair specifically for this conversation. Here's my public key.

Server Greetings, client. TLS version 1.3 suits me fine. Based on your cipher list, I've decided we'll use the Advanced Encryption Standard with Galois/Counter Mode (AES-GCM) cipher. I, too, created a new key pair for this conversation. Here is my public key and my certificate so you can prove that I truly am *google.com*. I'm also sending along a 32-byte value that corresponds to the TLS version you asked me to use. Finally, I'm including both a signature and a *message authentication code* (MAC) derived using your public key of everything we've discussed so far so you can verify the integrity of my reply when you receive it.

Client (to self) An authority I trust signed the server's certificate, so I'm confident I'm speaking to *google.com*. I've derived this conversation's symmetric key from the server's signature by using my private key. Using this symmetric key, I've verified the MAC and made sure no one has tampered with the server's reply. The 32 bytes in the reply

corresponds to TLS version 1.3, so no one is attempting to trick the server into using an older, weaker version of TLS. I now have everything I need to securely communicate with the server.

Client (to server) *Here is some encrypted data.*

The 32-byte value in the server's hello message prevents *downgrade attacks*, in which an attacker intercepts the client's hello message and modifies it to request an older, weaker version of TLS. If the client asked for TLS v1.3, but an attacker changed the client's hello message to ask for TLS v1.1, the 32-byte value in the server's hello message would correspond to TLS v1.1. When the client received the server's hello message, it would notice that the value indicated the wrong TLS version and abort the handshake.

From this point forward, the client and server communicate using AES-GCM symmetric-key cryptography (in this hypothetical example). Both the client and the server encapsulate application layer payloads in TLS records before passing the payloads onto the transport layer.

Despite its name, TLS is not a transport layer protocol. Instead, it's situated between the transport and application layers of the TCP/IP stack. TLS encrypts an application layer protocol's payload before passing the payload onto the transport layer. Once the payload reaches its destination, TLS receives the payload from the transport layer, decrypts it, and passes the payload along to the application layer protocol.

Forward Secrecy

The handshake method in our hypothetical conversation is an example of the Diffie-Hellman (DH) key exchange used in TLS v1.3. The *DH key exchange* calls for the creation of new client and server key pairs, and a new symmetric key, all of which should exist for only the duration of the session. Once a session ends, the client and server shall discard the session keys.

The use of per-session keys means that TLS v1.3 gives you *forward secrecy*; an attacker who compromises your session keys can compromise only the data exchanged during that session. An attacker cannot use those keys to decrypt data exchanged during any other session.

In Certificate Authorities We Trust

My father and I took a trip to Ireland shortly before I started authoring this book. In preparation for our adventure, I needed to obtain a new passport, since my old one had long since expired. The process was easy. I filled out an application, collected my vital records, took a picture of the bad side of my head, and presented everything, along with an application fee, to my local US Post Office branch. I also attested I was myself to the notary. A few weeks later, I received a newly minted US passport in the mail.

When we arrived in Ireland, a lovely customs agent greeted us and requested our passports. She asked questions about our holiday as her computer authenticated our identities. After no more than three minutes, she returned our passports and welcomed us to Ireland.

My passport represents the US government's attestation that I am Adam Woodbeck. But it's only as good as Ireland's trust in the US government's ability to verify my identity. If Ireland doesn't trust the United States, it will not take the United States' word that I am me and will most likely refuse to let me enter the country. (If I'm being honest, I'm not charming enough to convince the customs agent to let me in on my word alone.)

TLS's certificates work in much the same way as my passport. If I wanted a new TLS certificate for *woodbeck.net*, I would send a request to a certificate authority, such as Let's Encrypt. The certificate authority would then verify I am the proper owner of *woodbeck.net*. Once satisfied, the certificate authority would issue a new certificate for *woodbeck.net* and cryptographically sign it with its certificate. My server can present this certificate to clients so they can authenticate my server by confirming the certificate authority's signature, giving them the confidence that they're communicating with the real *woodbeck.net*, not an impostor.

A certificate authority issuing a signed certificate for *woodbeck.net* is analogous to the US government issuing my passport. They are both issued by trusted institutions that attest to their subject's authenticity. Like Ireland's trust of the United States, clients are inclined to trust the *woodbeck.net* certificate only if they trust the certificate authority that signed it. I could create my own certificate authority and self-sign certificates as easy as I could create a document claiming to be my passport. But Ireland would sooner admit that Jack Daniel's Tennessee Whiskey is superior to Jameson Irish Whiskey than trust my self-issued passport, and no operating system or web browser in the world would trust my self-signed certificate.

How to Compromise TLS

On December 24, 2013, Google learned that the Turktrust certificate authority in Turkey had mistakenly issued a certificate that allowed a malicious actor to masquerade as *google.com*. This meant that attackers could fool your web browser into thinking it was talking to Google over a TLS connection and trick you into divulging your credentials. Google quickly noticed the mistake and took steps to remedy the situation.

Turktrust's mess-up undermined its authority and compromised our trust. But even if the certificate authorities operate correctly, attackers can narrow their focus and target individuals instead. If an attacker were able to install his own CA certificate in your operating system's trusted certificate storage, your computer would trust any certificate he signs. This means an attacker could compromise all your TLS traffic.

Most people don't get this kind of special attention. Instead, an attacker is more likely to compromise a server. Once compromised, the attacker could capture all TLS traffic and the corresponding session keys from memory.

You're unlikely to encounter any of these scenarios, but it's important to be aware that they are possible. Overall, TLS 1.3 offers excellent security and is tough to compromise because of its full handshake signature, downgrade protection, forward secrecy, and strong encryption.

Protecting Data in Transit

Ensuring the integrity of the data you transmit over a network should be your primary focus, no matter whether it's your own data or the data of others. Go makes using TLS so easy that you would have a tough time justifying not using it. In this section, you'll learn how to add TLS support to both the client and the server. You'll also see how TLS works over TCP and how to mitigate the threat of malicious certificates with certificate pinning.

Client-side TLS

The client's primary concern during the handshake process is to authenticate the server by using its certificate. If the client cannot trust the server, it cannot consider its communication with the server secure. The `net/http`/`httptest` package provides constructs that easily demonstrate Go's HTTP-over-TLS support (see Listing 11-1).

```
package ch11

import (
    "crypto/tls"
    "net"
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"
    "time"

    "golang.org/x/net/http2"
)

func TestClientTLS(t *testing.T) {
    ts := ❶httptest.NewTLSServer(
        http.HandlerFunc(
            func(w http.ResponseWriter, r *http.Request) {
                if ❷r.TLS == nil {
                    u := "https://" + r.Host + r.RequestURI
                    http.Redirect(w, r, u, http.StatusMovedPermanently)
                    return
                }
                w.WriteHeader(http.StatusOK)
            },
        ),
    )
    defer ts.Close()

    resp, err := ❸ts.Client().Get(ts.URL)
    if err != nil {
        t.Fatal(err)
    }
}
```

```

if resp.StatusCode != http.StatusOK {
    t.Errorf("expected status %d; actual status %d",
        http.StatusOK, resp.StatusCode)
}

```

Listing 11-1: Testing HTTPS client and server support (tls_client_test.go)

The `httptest.NewTLSServer` function returns an HTTPS server ❶. Aside from the function name, this bit of code looks identical to our use of `httptest` in Chapter 8. Here, the `httptest.NewTLSServer` function handles the HTTPS server's TLS configuration details, including the creation of a new certificate. No trusted authority signed this certificate, so no discerning HTTPS client would trust it. You'll see how to work around this detail in just a moment by using a preconfigured client.

If the server receives the client's request over HTTP, the request's TLS field will be `nil`. You can check for this case ❷ and redirect the client to the HTTPS endpoint accordingly.

For testing purposes, the server's `Client` method ❸ returns a new `*http.Client` that inherently trusts the server's certificate. You can use this client to test TLS-specific code within your handlers.

Let's see what happens in Listing 11-2 when you attempt to communicate with the same server by using a new client without inherent trust for the server's certificate.

--snip--

```

tp := &http.Transport{
    TLSClientConfig: &tls.Config{
        CurvePreferences: []tls.CurveID{❶tls.CurveP256},
        MinVersion:       tls.VersionTLS12,
    },
}

err = ❷http2.ConfigureTransport(tp)
if err != nil {
    t.Fatal(err)
}

client2 := &http.Client{Transport: tp}

_, err = client2.Get(ts.URL)
if err == nil || !strings.Contains(err.Error(),
    "certificate signed by unknown authority") {
    t.Fatalf("expected unknown authority error; actual: %q", err)
}

❸ tp.TLSClientConfig.InsecureSkipVerify = true

resp, err = client2.Get(ts.URL)
if err != nil {
    t.Fatal(err)
}

```

```

    if resp.StatusCode != http.StatusOK {
        t.Errorf("expected status %d; actual status %d",
            http.StatusOK, resp.StatusCode)
    }
}

```

Listing 11-2: Testing the HTTPS server with a discerning client (`tls_client_test.go`)

You override the default TLS configuration in your client's transport by creating a new transport, defining its TLS configuration, and configuring `http2` to use this transport. It's good practice to restrict your client's curve preference to the P-256 curve ❶ and avoid the use of P-384 and P-521. P-256 is immune to timing attacks, whereas P-384 and P-521 are not. Also, your client will negotiate a minimum of TLS 1.2.

An *elliptic curve* is a plane curve in which all points along the curve satisfy the same polynomial equation. Whereas first-generation cryptography like RSA uses large prime numbers to derive keys, elliptic curve cryptography uses points along an elliptic curve for key generation. P-256, P-384, and P-521 are specific elliptic curves defined in the National Institute of Standards and Technology's Digital Signature Standard. You can find more details in the Federal Information Processing Standards (FIPS) publication 186-4 (<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf>).

Since your transport no longer relies on the default TLS configuration, the client no longer has inherent HTTP/2 support. You need to explicitly bless your transport with HTTP/2 support ❷ if you want to use it. Of course, this test doesn't rely on HTTP/2, but this implementation detail can trip you up if you're unaware that overriding the transport's TLS configuration removes HTTP/2 support.

Your client uses the operating system's trusted certificate store because you don't explicitly tell it which certificates to trust. The first call to the test server results in an error because your client doesn't trust the server certificate's signatory. You could work around this and configure your client's transport to skip verification of the server's certificate by setting its `InsecureSkipVerify` field to `true` ❸. I don't recommend you entertain enabling `InsecureSkipVerify` for anything other than debugging. Shipping code with this enabled is a code smell in my opinion. You'll learn a better alternative later in this chapter when we discuss a concept known as *certificate pinning*. As the field name implies, enabling it makes your client inherently insecure and susceptible to man-in-the-middle attacks, since it now blindly trusts any certificate a server offers up. If you make the same call with your newly naive client, you'll see that it happily negotiates TLS with the server.

TLS over TCP

TLS is stateful; a client and a server negotiate session parameters during the initial handshake only, and once they've agreed, they exchange encrypted TLS records for the duration of the session. Since TCP is also stateful, it's the ideal transport layer protocol with which to implement TLS, because you can leverage TCP's reliability guarantees to maintain your TLS sessions.

Let's take the application protocol out of the picture for a moment and learn how to establish a TLS connection over TCP. Listing 11-3 demonstrates how to use the `crypto/tls` package to initiate a TLS connection with a few lines of code.

```
--snip--

func TestClientTLSGoogle(t *testing.T) {
    conn, err := ❶tls.DialWithDialer(
        &net.Dialer{Timeout: 30 * time.Second},
        "tcp",
        "www.google.com:443",
        &tls.Config{
            CurvePreferences: []tls.CurveID{tls.CurveP256},
            MinVersion:         tls.VersionTLS12,
        },
    )
    if err != nil {
        t.Fatal(err)
    }

    state := ❷conn.ConnectionState()
    t.Logf("TLS 1.%d", state.Version-tls.VersionTLS10)
    t.Log(tls.CipherSuiteName(state.CipherSuite))
    t.Log(state.VerifiedChains[0][0].Issuer.Organization[0])

    _ = conn.Close()
}
```

Listing 11-3: Starting a TLS connection with `www.google.com` (`tls_client_test.go`)

The `tls.DialWithDialer` function ❶ accepts a `*net.Dialer`, a network, an address, and a `*tls.Config`. Here, you give your dialer a time-out of 30 seconds and specify recommended TLS settings. If successful, you can inspect the connection's state ❷ to glean details about your TLS connection.

Listing 11-4 shows the output of Listing 11-3's test.

```
$ go test -race -run TestClientTLSGoogle -v ./...
=== RUN   TestClientTLSGoogle
    TestClientTLSGoogle: tls_client_test.go:89: TLS 1.3
    TestClientTLSGoogle: tls_client_test.go:90: TLS_AES_128_GCM_SHA256
    TestClientTLSGoogle: tls_client_test.go:91: Google Trust Services
--- PASS: TestClientTLSGoogle (0.31s)
PASS
```

Listing 11-4: Running the `TestClientTLSGoogle` test

Your TLS client is using the `TLS_AES_128_GCM_SHA256` cipher suite over TLS version 1.3. Notice that `tls.DialWithDialer` did not object to the server's certificate. The underlying TLS client used the operating system's trusted certificate storage and confirmed that `www.google.com`'s certificate is signed by a trusted CA—Google Trust Services, in this example.

Server-side TLS

The server-side code isn't much different from what you've learned thus far. The main difference is that the server needs to present a certificate to the client as part of the handshake process. You can create one with the *generate_cert.go* file found in Go's *src/crypto/tls* subdirectory. For production use, you're better off using certificates from Let's Encrypt or another certificate authority. You can use the LEGO library (<https://github.com/go-acme/lego/>) to add certificate management to your services. Generate a new cert and private key, like so:

```
$ go run $GOROOT/src/crypto/tls/generate_cert.go -host localhost -ecdsa-curve P256
```

This command creates a certificate named *cert.pem* with the hostname *localhost* and a private key named *key.pem*. The rest of the code in this section assumes that both files exist in the current directory.

Keeping with the tradition of earlier chapters, Listing 11-5 includes the first bit of code for a TLS-only echo server.

```
package ch11

import (
    "context"
    "crypto/tls"
    "fmt"
    "net"
    "time"
)

func NewTLSServer(ctx context.Context, address string,
    maxIdle time.Duration, tlsConfig *tls.Config) *Server {
    return &Server{
        ctx:      ctx,
        ready:    make(chan struct{}),
        addr:     address,
        maxIdle:  maxIdle,
        tlsConfig: tlsConfig,
    }
}

type Server struct {
    ctx      context.Context
    ready    chan struct{}

    addr      string
    maxIdle   time.Duration
    tlsConfig *tls.Config
}

func (s *Server) Ready() {
    if s.ready != nil {
        <-s.ready
    }
}
```

Listing 11-5: Server struct type and constructor function (*tls_echo.go*)

The `Server` struct has a few fields used to record its settings, its TLS configuration, and a channel to signal when the server is ready for incoming connections. You'll write a test case and use the `Ready` method ❶ a little later in this section to block until the server is ready to accept connections.

The `NewTLSServer` function accepts a context for stopping the server, an address, the maximum duration the server should allow connections to idle, and a TLS configuration. Although controlling for idling clients isn't related to TLS, you'll use the maximum idle duration to push the socket deadline forward, as in Chapter 3.

Servers you used in earlier chapters rely on the separate concepts of listening and serving. Often, you'll invoke a helper function that will do both for you, such as the `net/http` server's `ListenAndServe` method. Listing 11-6 adds a similar method to the echo server.

--snip--

```
func (s *Server) ListenAndServeTLS(certFn, keyFn string) error {
    if s.addr == "" {
        s.addr = "localhost:443"
    }

    l, err := net.Listen("tcp", s.addr)
    if err != nil {
        return fmt.Errorf("binding to tcp %s: %w", s.addr, err)
    }

    if s.ctx != nil {
        go func() {
            <-s.ctx.Done()
            _ = l.Close()
        }()
    }

    return s.ServeTLS(l, certFn, keyFn)
}
```

Listing 11-6: Adding methods to listen and serve and signal the server's readiness for connections (tls_echo.go)

The `ListenAndServe` method accepts full paths to a certificate and a private key and returns an error. It creates a new `net.Listener` bound to the server's address and then spins off a goroutine to close the listener when you cancel the context. Finally, the method passes the listener, the certificate path, and the key path onto the server's `ServeTLS` method.

Listing 11-7 rounds out the echo server's implementation with its `ServeTLS` method.

--snip--

```
func (s Server) ServeTLS(l net.Listener, certFn, keyFn string) error {
    if s.tlsConfig == nil {
        s.tlsConfig = &tls.Config{
```



```

        CurvePreferences:      []tls.CurveID{tls.CurveP256},
        MinVersion:            tls.VersionTLS12,
        ❶ PreferServerCipherSuites: true,
    }
}

if len(s.tlsConfig.Certificates) == 0 &&
s.tlsConfig.GetCertificate == nil {
    cert, err := ❷tls.LoadX509KeyPair(certFn, keyFn)
    if err != nil {
        return fmt.Errorf("loading key pair: %v", err)
    }

    s.tlsConfig.Certificates = []tls.Certificate{cert}
}

tlsListener := ❸tls.NewListener(l, s.tlsConfig)
if s.ready != nil {
    close(s.ready)
}

```

Listing 11-7: Adding TLS support to a net.Listener (tls_echo.go)

The ServeTLS method first checks the server's TLS configuration. If it's nil, it adds a default configuration with PreferServerCipherSuites set to true ❶. PreferServerCipherSuites is meaningful to the server only, and it makes the server use its preferred cipher suite instead of deferring to the client's preference.

If the server's TLS configuration does not have at least one certificate, or if its GetCertificate method is nil, you create a new tls.Certificate by reading in the certificate and private-key files from the filesystem ❷.

At this point in the code, the server has a TLS configuration with at least one certificate ready to present to clients. All that's left is to add TLS support to the net.Listener by passing it and the server's TLS configuration to the tls.NewListener function ❸. The tls.NewListener function acts like middleware, in that it augments the listener to return TLS-aware connection objects from its Accept method.

Listing 11-8 finishes up the ServeTLS method by accepting connections from the listener and handling them in separate goroutines.

--snip--

```

for {
    conn, err := ❶tlsListener.Accept()
    if err != nil {
        return fmt.Errorf("accept: %v", err)
    }

    go func() {
        defer func() { _ = ❷conn.Close() }()

        for {
            if s.maxIdle > 0 {

```

```

        err := ❸conn.SetDeadline(time.Now().Add(s.maxIdle))
        if err != nil {
            return
        }
    }

    buf := make([]byte, 1024)
    n, err := ❹conn.Read(buf)
    if err != nil {
        return
    }

    _, err = conn.Write(buf[:n])
    if err != nil {
        return
    }
}

}()
}
}

```

Listing 11-8: Accepting TLS-aware connections from the listener (tls_echo.go)

This pattern is like the one you’ve seen in earlier chapters. You use an endless for loop to continually block on the listener’s `Accept` method ❶, which returns a new `net.Conn` object when a client successfully connects. Since you’re using a TLS-aware listener, it returns connection objects with underlying TLS support. You interact with these connection objects the same as you always do. `Go` abstracts the TLS details away from you at this point. You then spin off this connection into its own goroutine to handle the connection from that point forward.

The server handles each connection the same way. It first conditionally sets the socket deadline to the server’s maximum idle duration ❸, then waits for the client to send data. If the server doesn’t read anything from the socket before it reaches the deadline, the connection’s `Read` method ❹ returns an *I/O time-out* error, ultimately causing the connection to close ❷.

If, instead, the server reads data from the connection, it writes that same payload back to the client. Control loops back around to reset the deadline and then wait for the next payload from the client.

Certificate Pinning

Earlier in the chapter, we discussed ways to compromise the trust that TLS relies on, whether by a certificate authority issuing fraudulent certificates or an attacker injecting a malicious certificate into your computer’s trusted certificate storage. You can mitigate both attacks by using certificate pinning.

Certificate pinning is the process of scrapping the use of the operating system’s trusted certificate storage and explicitly defining one or more trusted certificates in your application. Your application will trust connections only from hosts presenting a pinned certificate or a certificate signed

by a pinned certificate. If you plan on deploying clients in zero-trust environments that must securely communicate with your server, consider pinning your server's certificate to each client.

Assuming the server introduced in the preceding section uses the *cert.pem* and the *key.pem* you generated for the hostname *localhost*, all clients will abort the TLS connection as soon as the server presents its certificate. Clients won't trust the server's certificate because no trusted certificate authority signed it.

You could set the `tls.Config`'s `InsecureSkipVerify` field to `true`, but as this method is insecure, I don't recommend you consider it a practical choice. Instead, let's explicitly tell our client it can trust the server's certificate by pinning the server's certificate to the client. Listing 11-9 has the beginnings of a test to show that process.

```
package ch11

import (
    "bytes"
    "context"
    "crypto/tls"
    "crypto/x509"
    "io"
    "io/ioutil"
    "strings"
    "testing"
    "time"
)

func TestEchoServerTLS(t *testing.T) {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    serverAddress := "localhost:34443"
    maxIdle := time.Second
    server := NewTLSServer(ctx, serverAddress, maxIdle, nil)
    done := make(chan struct{})

    go func() {
        err := ❶server.ListenAndServeTLS("cert.pem", "key.pem")
        if err != nil && !strings.Contains(err.Error(),
            "use of closed network connection") {
            t.Error(err)
            return
        }
        done <- struct{}{}
    }()
    ❷server.Ready()
```

Listing 11-9: Creating a new TLS echo server and starting it in the background (tls_echo_test.go)

Since the hostname in *cert.pem* is *localhost*, you create a new TLS echo server listening on *localhost* port 34443. The port isn't important here, but clients expect the server to be reachable by the same hostname as the one

in the certificate it presents. You spin up the server in the background by using the *cert.pem* and *key.pem* files ❶ and block until it's ready for incoming connections ❷.

Listing 11-10 picks up where we left off by creating a client TLS configuration with explicit trust for the server's certificate.

--snip--

```
cert, err := ioutil.ReadFile("cert.pem")
if err != nil {
    t.Fatal(err)
}

certPool := ❶x509.NewCertPool()
if ok := certPool.AppendCertsFromPEM(cert); !ok {
    t.Fatal("failed to append certificate to pool")
}

tlsConfig := &tls.Config{
    CurvePreferences: []tls.CurveID{tls.CurveP256},
    MinVersion:       tls.VersionTLS12,
    ❷ RootCAs:         certPool,
}
```

Listing 11-10: Pinning the server certificate to the client (tls_echo_test.go)

Pinning a server certificate to the client is straightforward. First, you read in the *cert.pem* file. Then, you create a new certificate pool ❶ and append the certificate to it. Finally, you add the certificate pool to the `tls.Config`'s `RootCAs` field ❷. As the name suggests, you can add more than one trusted certificate to the certificate pool. This can be useful when you are migrating to a new certificate but have yet to completely phase out the old certificate.

The client, using this configuration, will authenticate only servers that present the *cert.pem* certificate or any certificate signed by it. Let's confirm this behavior in the rest of the test (see Listing 11-11).

--snip--

```
conn, err := ❶tls.Dial("tcp", serverAddress, tlsConfig)
if err != nil {
    t.Fatal(err)
}

hello := []byte("hello")
_, err = conn.Write(hello)
if err != nil {
    t.Fatal(err)
}

b := make([]byte, 1024)
n, err := conn.Read(b)
if err != nil {
```

```

        t.Fatal(err)
    }

    if actual := b[:n]; !bytes.Equal(hello, actual) {
        t.Fatalf("expected %q; actual %q", hello, actual)
    }

    ❷ time.Sleep(2 * maxIdle)
    _, err = conn.Read(b)
    if err != ❸ io.EOF {
        t.Fatal(err)
    }

    err = conn.Close()
    if err != nil {
        t.Fatal(err)
    }

    cancel()
    <-done
}

```

Listing 11-11: Authenticating the server by using a pinned certificate (tls_echo_test.go)

You pass `tls.Dial` the `tls.Config` with the pinned server certificate ❶. Your TLS client authenticates the server's certificate without having to resort to using `InsecureSkipVerify` and all the insecurity that option introduces.

Now that you've set up a trusted connection with a server, even though the server presented an unsigned certificate, let's make sure the server works as expected. It should echo back any message you send it. If you idle long enough ❷, you find that your next interaction with the socket results in an error ❸, showing the server closed the socket.

Mutual TLS Authentication

In the preceding section, you learned how clients authenticate servers by using the server's certificate and a trusted third-party certificate or by configuring the client to explicitly trust the server's certificate. Servers can authenticate clients in the same manner. This is particularly useful in zero-trust network infrastructures, where clients and servers must each prove their identities. For example, you may have a client outside your network that must present a certificate to a proxy before the proxy will allow the client to access your trusted network resources. Likewise, the client authenticates the certificate presented by your proxy to make sure it's talking to your proxy and not one controlled by a malicious actor.

You can instruct your server to set up TLS sessions with only authenticated clients. Those clients would have to present a certificate signed by a trusted certificate authority or pinned to the server. Before you can look at example code, the client needs a certificate it can present to the server for

authentication. However, clients cannot use the certificates generated with `$GOROOT/src/crypto/tls/generate_cert.go` for client authentication. Instead, you need to create your own certificate and private key.

Generating Certificates for Authentication

Go's standard library contains everything you need to generate your own certificates using the elliptic curve digital signature algorithm (ECDSA) and the P-256 elliptic curve. Listing 11-12 shows the beginnings of a command line utility for doing exactly that. As you go through it, keep in mind that it may not entirely fit your use case. For example, it creates 10-year certificates and uses my name as the certificate's subject, which you likely don't want to use in your code (though if you do, I'm flattered). Tweak, as necessary.

```
package main

import (
    "crypto/ecdsa"
    "crypto/elliptic"
    "crypto/rand"
    "crypto/x509"
    "crypto/x509/pkix"
    "encoding/pem"
    "flag"
    "log"
    "math/big"
    "net"
    "os"
    "strings"
    "time"
)

var (
    host = flag.String("host", "localhost",
        "Certificate's comma-separated host names and IPs")
    certFn = flag.String("cert", "cert.pem", "certificate file name")
    keyFn  = flag.String("key", "key.pem", "private key file name")
)

func main() {
    flag.Parse()

    serial, err := rand.Int(rand.Reader, new(big.Int).Lsh(big.NewInt(1),
        128))
    if err != nil {
        log.Fatal(err)
    }

    notBefore := time.Now()
    template := x509.Certificate{
        SerialNumber: serial,
```

```

Subject: pkix.Name{
    Organization: []string{"Adam Woodbeck"},
},
NotBefore: notBefore,
NotAfter:  notBefore.Add(10 * 356 * 24 * time.Hour),
KeyUsage: x509.KeyUsageKeyEncipherment |
           x509.KeyUsageDigitalSignature |
           x509.KeyUsageCertSign,
ExtKeyUsage: []x509.ExtKeyUsage{
    x509.ExtKeyUsageServerAuth,
    ❷x509.ExtKeyUsageClientAuth,
},
BasicConstraintsValid: true,
IsCA:                  true,
}

```

Listing 11-12: Creating an X.509 certificate template (cert/generate.go)

The command line utility accepts a comma-separated list of hostnames and IP addresses that will use the certificate. It also allows you to specify the certificate and private-key filenames, but it defaults to our familiar *cert.pem* and *key.pem* filenames.

The process of generating a certificate and a private key involves building a template in your code that you then encode to the X.509 format. Each certificate needs a serial number, which a certificate authority typically assigns. Since you're generating your own self-signed certificate, you generate your own serial number using a cryptographically random, unsigned 128-bit integer ❶. You then create an `x509.Certificate` object that represents an X.509-formatted certificate and set various values, such as the serial number, the certificate's subject, the validity lifetime, and various usages for this certificate. Since you want to use this certificate for client authentication, you must include the `x509.ExtKeyUsageClientAuth` value ❷. If you omit this value, the server won't be able to verify the certificate when presented by the client.

The template is almost ready. You just need to add the hostnames and IP addresses before generating the certificate (see Listing 11-13).

--snip--

```

for _, h := range ❶strings.Split(*host, ",") {
    if ip := net.ParseIP(h); ip != nil {
        ❷ template.IPAddresses = append(template.IPAddresses, ip)
    } else {
        ❸ template.DNSNames = append(template.DNSNames, h)
    }
}

priv, err := ❹ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
if err != nil {
    log.Fatal(err)
}

```

```

der, err := ❸x509.CreateCertificate(rand.Reader, &template,
    &template, &priv.PublicKey, priv)
if err != nil {
    log.Fatal(err)
}

cert, err := os.Create(*certFn)
if err != nil {
    log.Fatal(err)
}

err = ❹pem.Encode(cert, &pem.Block{Type: "CERTIFICATE", Bytes: der})
if err != nil {
    log.Fatal(err)
}

if err := cert.Close(); err != nil {
    log.Fatal(err)
}
log.Println("wrote", *certFn)

```

Listing 11-13: Writing the Privacy-Enhanced Mail (PEM)-encoded certificate (cert/generate.go)

You loop through the comma-separated list of hostnames and IP addresses ❶, assigning each to its appropriate slice in the template. If the hostname is an IP address, you assign it to the `IPAddresses` slice ❷. Otherwise, you assign the hostname to the `DNSNames` slice ❸. Go's TLS client uses these values to authenticate a server. For example, if the client connects to `https://www.google.com` but the common name or alternative names in the server's certificate do not match `www.google.com`'s hostname or resolved IP address, the client fails to authenticate the server.

Next you generate a new ECDSA private key ❹ using the P-256 elliptic curve. At this point, you have everything you need to generate the certificate. The `x509.CreateCertificate` function ❺ accepts a source of entropy (crypto/rand's `Reader` is ideal), the template for the new certificate, a parent certificate, a public key, and a corresponding private key. It then returns a slice of bytes containing the Distinguished Encoding Rules (DER)-encoded certificate. You use your template for the parent certificate since the resulting certificate signs itself. All that's left to do is create a new file, generate a new `pem.Block` with the DER-encoded byte slice, and PEM-encode everything to the new file ❻. You don't have to concern yourself with the various encodings. Go is quite happy with using PEM-encoded certificates on disk.

Now that you have a new certificate on disk, let's write the corresponding private key in Listing 11-14.

--snip--

```

key, err := os.OpenFile(*keyFn, os.O_WRONLY|os.O_CREATE|os.O_TRUNC,
    ❶0600)
if err != nil {
    log.Fatal(err)
}

```



```

    }

    privKey, err := ❷x509.MarshalPKCS8PrivateKey(priv)
    if err != nil {
        log.Fatal(err)
    }

    err = ❸pem.Encode(key, &pem.Block{Type: "EC PRIVATE KEY",
    Bytes: privKey})
    if err != nil {
        log.Fatal(err)
    }

    if err := key.Close(); err != nil {
        log.Fatal(err)
    }
    log.Println("wrote", *keyFn)
}

```

Listing 11-14: Writing the PEM-encoded private key (cert/generate.go)

Whereas the certificate is meant to be publicly shared, the private key is just that: private. You should take care to assign it minimal permissions. Here, you're giving only the user read-write access to the private-key file ❶ and removing access for everyone else. We marshal the private key into a byte slice ❷ and, similarly, assign it to a new `pem.Block` before writing the PEM-encoded output to the private-key file ❸.

Listing 11-15 uses the preceding code to generate certificate and key pairs for the server and the client.

```

$ go run cert/generate.go -cert serverCert.pem -key serverKey.pem -host
localhost
2006/01/02 15:04:05 wrote serverCert.pem
2006/01/02 15:04:05 wrote serverKey.pem
$ go run cert/generate.go -cert clientCert.pem -key clientKey.pem -host
localhost
2006/01/02 15:04:05 wrote clientCert.pem
2006/01/02 15:04:05 wrote clientKey.pem

```

Listing 11-15: Generating a certificate and private-key pair for the server and the client

Since the server binds to `localhost` and the client connects to the server from `localhost`, this value is appropriate for both the client and server certificates. If you want to move the client to a different hostname or bind the server to an IP address, for example, you'll need to change the `host` flag accordingly.

Implementing Mutual TLS

Now that you've generated certificate and private-key pairs for both the server and the client, you can start writing their code. Let's write a test that implements mutual TLS authentication between our echo server and a client, starting in Listing 11-16.

```

package ch11

import (
    "bytes"
    "context"
    "crypto/tls"
    "crypto/x509"
    "errors"
    "io/ioutil"
    "strings"
    "testing"
)

func caCertPool(caCertFn string) (*x509.CertPool, error) {
    caCert, err := ❶ ioutil.ReadFile(caCertFn)
    if err != nil {
        return nil, err
    }

    certPool := x509.NewCertPool()
    if ok := ❷ certPool.AppendCertsFromPEM(caCert); !ok {
        return nil, errors.New("failed to add certificate to pool")
    }

    return certPool, nil
}

```

Listing 11-16: Creating a certificate pool to serve CA certificates (tls_mutual_test.go)

Both the client and server use the `caCertPool` function to create a new X.509 certificate pool. The function accepts the file path to a PEM-encoded certificate, which you read in ❶ and append to the new certificate pool ❷. The certificate pool serves as a source of trusted certificates. The client puts the server's certificate in its certificate pool, and vice versa.

Listing 11-17 details the initial test code to demonstrate mutual TLS authentication between a client and a server.

```

--snip--

func TestMutualTLSAuthentication(t *testing.T) {
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    serverPool, err := caCertPool(❶ "clientCert.pem")
    if err != nil {
        t.Fatal(err)
    }

    cert, err := ❷ tls.LoadX509KeyPair("serverCert.pem", "serverKey.pem")
    if err != nil {
        t.Fatalf("loading key pair: %v", err)
    }
}

```

Listing 11-17: Instantiating a CA cert pool and a server certificate (tls_mutual_test.go)

Before creating the server, you need to first populate a new CA certificate pool with the client's certificate ❶. You also need to load the server's certificate at this point ❷ instead of relying on the server's `ServeTLS` method to do it for you, as you have in previous listings. Why you need the server's certificate now will be clear when you see the TLS configuration changes in Listing 11-18.

--snip--

```
serverConfig := &tls.Config{
    Certificates: []tls.Certificate{cert},
    ❶ GetConfigForClient: func(hello *tls.ClientHelloInfo) (*tls.Config,
        error) {
        return &tls.Config{
            Certificates:          []tls.Certificate{❷cert},
            ❸ ClientAuth:            tls.RequireAndVerifyClientCert,
            ❹ ClientCAs:             serverPool,
            CurvePreferences:     []tls.CurveID{tls.CurveP256},
            MinVersion:             ❺tls.VersionTLS13,
            PreferServerCipherSuites: true,
```

Listing 11-18: Accessing the client's hello information using `GetConfigForClient` (`tls_mutual_test.go`)

Remember that in Listing 11-13, you defined the `IPAddresses` and `DNSNames` slices of the template used to generate your client's certificate. These values populate the common name and alternative names portions of the client's certificate. You learned that Go's TLS client uses these values to authenticate the server. But the server does not use these values from the client's certificate to authenticate the client.

Since you're implementing mutual TLS authentication, you need to make some changes to the server's certificate verification process so that it authenticates the client's IP address or hostnames against the client certificate's common name and alternative names. To do that, the server at the very least needs to know the client's IP address. The only way you can get client connection information before certificate verification is by defining the `tls.Config`'s `GetConfigForClient` method ❶. This method allows you to define a function that receives the `*tls.ClientHelloInfo` object created as part of the TLS handshake process with the client. From this, you can retrieve the client's IP address. But first, you need to return a proper TLS configuration.

You add the server's certificate to the TLS configuration ❷ and the server pool to the TLS configuration's `ClientCAs` field ❹. This field is the server's equivalent to the TLS configuration's `RootCAs` field on the client. You also need to tell the server that every client must present a valid certificate before completing the TLS handshake process ❸. Since you control both the client and the server, specify a minimum TLS protocol version of 1.3 ❺.

This function returns the same TLS configuration for every client connection. As mentioned, the only reason you're using the `GetConfigForClient` method is so you can retrieve the client's IP from its hello information.

Listing 11-19 implements the verification process that authenticates the client by using its IP address and its certificate’s common name and alternative names.

--snip--

```

❶ VerifyPeerCertificate: func(rawCerts [][]byte,
    verifiedChains [][]*x509.Certificate) error {

    opts := x509.VerifyOptions{
        KeyUsages: []x509.ExtKeyUsage{
            ❷ x509.ExtKeyUsageClientAuth,
        },
        Roots: ❸ serverPool,
    }

    ip := strings.Split(hello.Conn.RemoteAddr().String(),
        ":")[0]
    hostnames, err := ❹ net.LookupAddr(ip)
    if err != nil {
        t.Errorf("PTR lookup: %v", err)
    }
    hostnames = append(hostnames, ip)

    for _, chain := range verifiedChains {
        opts.Intermediates = x509.NewCertPool()
        for _, cert := range ❺ chain[1:] {
            opts.Intermediates.AddCert(cert)
        }

        for _, hostname := range hostnames {
            opts.DNSName = ❻ hostname
            _, err = chain[0].Verify(opts)
            if err == nil {
                return nil
            }
        }
    }

    return errors.New("client authentication failed")
},
}, nil
}

```

Listing 11-19: Making the server authenticate the client’s IP and hostnames (tls_mutual_test.go)

Since you want to augment the usual certificate verification process on the server, you define an appropriate function and assign it to the TLS configuration’s `VerifyPeerCertificate` method ❶. The server calls this method after the normal certificate verification checks. The only check you’re performing above and beyond the normal checks is to verify the client’s hostname with the leaf certificate.

The *leaf certificate* is the last certificate in the certificate chain given to the server by the client. The leaf certificate contains the client's public key. All other certificates in the chain are intermediate certificates used to verify the authenticity of the leaf certificate and culminate with the certificate authority's certificate. You'll find each leaf certificate at index 0 in each `verifiedChains` slice. In other words, you can find the leaf certificate of the first chain at `verifiedChains[0][0]`. If the server calls your function assigned to the `VerifyPeerCertificate` method, the leaf certificate in the first chain exists at a minimum.

Create a new `x509.VerifyOptions` object and modify the `KeyUsages` method to indicate you want to perform client authentication ❷. Then, assign the server pool to the `Roots` method ❸. The server uses this pool as its trusted certificate source during verification.

Now, extract the client's IP address from the connection object in the `*tls.ClientHelloInfo` object named `hello` passed into Listing 11-18's `GetConfigForClient` method. Use the IP address to perform a reverse DNS lookup ❹ to consider any hostnames assigned to the client's IP address. If this lookup fails or returns an empty slice, the way you handle that situation is up to you. If you're relying on the client's hostname for authentication and the reverse lookup fails, you cannot authenticate the client. But if you're using the client's IP address only in the certificate's common name or alternative names, then a reverse lookup failure is inconsequential. For demonstration purposes, we'll consider a failed reverse lookup to equate to a failed test. At minimum, you append the client's IP address to the `hostnames` slice.

All that's left to do is loop through each verified chain, assign a new intermediate certificate pool to `opts.Intermediates`, add all certificates but the leaf certificate to the intermediate certificate pool ❺, and attempt to verify the client ❻. If verification returns a `nil` error, you authenticated the client. If you fail to verify each hostname with each leaf certificate, return an error to indicate that client authentication failed. The client will receive an error, and the server will terminate the connection.

Now that the server's TLS configuration properly authenticates client certificates, continue with the server implementation in Listing 11-20.

--snip--

```
serverAddress := "localhost:4443"
server := NewTLSServer(ctx, serverAddress, 0, ❶serverConfig)
done := make(chan struct{})

go func() {
    err := server.ListenAndServeTLS("serverCert.pem", "serverKey.pem")
    if err != nil && !strings.Contains(err.Error(),
        "use of closed network connection") {
        t.Error(err)
        return
    }
}
```

```

        done <- struct{}{}
    }()
    ❷ server.Ready()

```

Listing 11-20: Starting the TLS server (tls_mutual_test.go)

Create a new TLS server instance, making sure to pass in the TLS configuration you just created ❶. Call its `ListenAndServeTLS` method in a goroutine and make sure to wait until the server is ready for connections ❷ before proceeding.

Now that the server implementation is ready, let's move on to the client portion of the test. Listing 11-21 implements a TLS client that can present *clientCert.pem* upon request by the server.

--snip--

```

clientPool, err := caCertPool(❶ "serverCert.pem")
if err != nil {
    t.Fatal(err)
}

clientCert, err := tls.LoadX509KeyPair("clientCert.pem", "clientKey.pem")
if err != nil {
    t.Fatal(err)
}

conn, err := tls.Dial("tcp", serverAddress, &tls.Config{
    ❷ Certificates:    []tls.Certificate{clientCert},
    CurvePreferences: []tls.CurveID{tls.CurveP256},
    MinVersion:       tls.VersionTLS13,
    ❸ RootCAs:        clientPool,
})
if err != nil {
    t.Fatal(err)
}

```

Listing 11-21: Pinning the server certificate to the client (tls_mutual_test.go)

The client retrieves a new certificate pool populated with the server's certificate ❶. The client then uses the certificate pool in the `RootCAs` field of its TLS configuration ❸, meaning the client will trust only server certificates signed by *serverCert.pem*. You also configure the client with its own certificate ❷ to present to the server upon request.

It's worth noting that the client and server have not initialized a TLS session yet. They haven't completed the TLS handshake. If `tls.Dial` returns an error, it isn't because of an authentication issue but more likely a TCP connection issue. Let's continue with the client code to initiate the handshake (see Listing 11-22).

--snip--

```

hello := []byte("hello")
_, err = conn.Write(hello)

```

```

    if err != nil {
        t.Fatal(err)
    }

    b := make([]byte, 1024)
    n, err := ❶ conn.Read(b)
    if err != nil {
        t.Fatal(err)
    }

    if actual := b[:n]; !bytes.Equal(hello, actual) {
        t.Fatalf("expected %q; actual %q", hello, actual)
    }

    err = conn.Close()
    if err != nil {
        t.Fatal(err)
    }

    cancel()
    <-done
}

```

Listing 11-22: TLS handshake completes as you interact with the connection
 (tls_mutual_test.go)

The first read from, or write to, the socket connection automatically initiates the handshake process between the client and the server. If the server rejects the client certificate, the read call ❶ will return a *bad certificate* error. But if you created appropriate certificates and properly pinned them, both the client and the server are happy, and this test passes.

What You’ve Learned

Transport Layer Security provides both authentication and encrypted communication between a client and a server. The server presents a certificate, signed by certificate authority, to a client as part of the TLS handshake process. The client verifies the certificate’s signatory. If a third party, trusted by the client, signed the server’s certificate, the server is authentic in the eyes of the client. From that point forward, the client and server communicate using symmetric-key cryptography.

By default, Go’s TLS configuration uses the operating system’s trusted certificate storage. This storage typically consists of certificates from the world’s foremost trusted certificate authorities. However, we can modify the TLS configuration to trust specific keys, a process known as key pinning.

We can also modify a server’s TLS configuration to require a certificate from the client. The server would then use this certificate to authenticate the client in the same manner the client authenticates the server. This process is known as mutual TLS authentication.

TLS 1.3 provides forward secrecy for all communication between a client and server. This means that compromising one session does not compromise any other session. Both the client and server generate per-session public- and private-key pairs. They also exchange an ephemeral shared secret as part of the handshake process. Once the session ends, the client and server shall purge the shared secret and their temporary key pairs. An attacker who was able to capture the shared secret and session traffic would be able to decrypt only that session's traffic. An attacker could not use the shared secret from one session to decrypt traffic from any other session.

Even though TLS is ubiquitous and secures much of the world's digital communication, attackers can compromise it. Part of a certificate authority's job is to verify that the entity requesting a certificate for a specific domain name owns the domain name. If attackers dupe a certificate authority, or the certificate authority otherwise makes a mistake and issues a fraudulent certificate, the owner of the fraudulent certificate could masquerade as Google, for example, and trick people into divulging sensitive information.

Another attack vector includes fooling a client into adding the attacker's certificate into the client's trusted certificate storage. The attacker could then issue and sign any certificate they want, and the client would inherently trust that the attacker is who their certificate claims them to be.

An attacker could also compromise a server and intercept TLS session keys and secrets, or even capture traffic at the application layer after the server has decrypted it.

Overall, however, these attacks are rare, and TLS succeeds at achieving its goals of authentication and encrypted communication.

PART IV

SERVICE ARCHITECTURE

12

DATA SERIALIZATION



A sizable portion of our work as developers involves integrating our network services with existing services, including legacy or third-party ones implemented in languages other than Go. These services must communicate by exchanging bytes of data in a way that is meaningful to both the sender and receiver, despite the different programming languages they're using. To do this, the sender converts data into bytes using a standard format and transfers the bytes over the network to the receiver. If the receiver understands the format used by the sender, it can convert the bytes back into structured data. This process of transforming structured data into successive bytes is known as *data serialization*.

Services can use data serialization to convert structured data to a series of bytes suitable for transport over a network or for persistence to storage. No matter whether the serialized data came from a network or disk, any code that understands the serialization format should be able to deserialize it to reconstruct a clone of the original object.

While writing this chapter, I initially struggled to explain the concept of data serialization. Then I realized we serialize data as we speak. Electrical impulses in my brain form words. My brain instructs my voice box to *serialize* these words into sound waves, which travel through the air and reach your ear. The sound waves vibrate your eardrum, which in turn transmits the vibrations to your inner ear. Hair-like structures in your inner ear deserialize these vibrations into electrical signals that your brain interprets as the original words I formed in my brain. We've just communicated using the serialization format of English, since it's a format we both understand.

You already have a bit of experience serializing data in your Go code too. The type-length-value binary encoding you learned in Chapter 4 and the JavaScript Object Notation (JSON) you posted over HTTP in Chapter 8 are examples of translating objects into well-known data serialization formats. We also PEM-encoded the certificates and private keys in Chapter 11 for persistence to disk.

This chapter will take a deeper dive into using data serialization for the purposes of storing data or sending between systems, which can make it accessible to services written in other languages. We could cover many data serialization formats, but we'll focus on the three that get the most use in Go network programming: JSON, protocol buffers, and Gob. We'll also spend some time on how to execute code on remote machines using a framework named gRPC. By the end of this chapter, you will know how to serialize data for storage or transmission and decode that data into meaningful data structures. You should be able to use techniques in this chapter to build services that can exchange complex data over a network or write code to communicate with existing network services.

Serializing Objects

Objects or structured data cannot traverse a network connection as is. In other words, you cannot pass in an object to `net.Conn's Write` method, since it accepts only a byte slice. Therefore, you need to serialize the object to a byte slice, which you can then pass to the `Write` method. Thankfully, Go makes this easy.

Go's standard library includes excellent support for popular data serialization formats in its encoding package. You've already used `encoding/binary` to serialize numbers into byte sequences, `encoding/json` to serialize objects into JSON for submission over HTTP, and `encoding/pem` to serialize TLS certificates and private keys to files. (Anytime you encounter a function or method whose name includes *encode* or *marshal*, it likely serializes data. Likewise, *decode* and *unmarshal* are synonymous with deserializing data.)

This section will build an application that serializes data into three binary encoding formats: JSON, protocol buffers, and Gob. Since I often have trouble keeping track of housework, the application will document chores to do around the house. The application's state will persist between executions because you don't want it to forget the chores when it exits. You'll serialize each task to a file and use your app to update it as needed.

To keep this program simple, you need a description of the chore and a way to determine whether it's complete. Listing 12-1 defines a new package with a type that represents a household chore.

```
package housework

type Chore struct {
    Complete bool
    Description string
}
```

Listing 12-1: A type to represent a household chore (housework/housework.go)

Go's JSON and Gob encoding packages can serialize exported struct fields only, so you define `Chore` as a struct, making sure to export its fields. The `Complete` field's value will be true if you've completed the chore. The `Description` field is the human-readable description of the chore.

You could use struct tags to instruct the encoders on how to treat each field, if necessary. For example, you could place the struct tag ``json:"-`"` on the `Complete` field to tell Go's JSON encoder to ignore this field instead of encoding it. Since you're perfectly happy to pass along all field values, you omit struct tags.

Once you've defined a chore structure, you can use it in an application that tracks chores on the command line. This application should show a list of chores and their status, allow you to add chores to the list, and mark chores as complete. Listing 12-2 includes the initial code for this housework application, including its command line usage details.

```
package main

import (
    "flag"
    "fmt"
    "log"
    "os"
    "path/filepath"
    "strconv"
    "strings"

    "github.com/awoodbeck/gnp/ch12/housework"
    ❶ storage "github.com/awoodbeck/gnp/ch12/json"
    // storage "github.com/awoodbeck/gnp/ch12/gob"
    // storage "github.com/awoodbeck/gnp/ch12/protobuf"
)
```

```

var dataFile string

func init() {
    flag.StringVar(&dataFile, "file", "housework.db", "data file")

    flag.Usage = func() {
        fmt.Fprintf(flag.CommandLine.Output(),
            ❷ `Usage: %s [flags] [add chore, ...|complete #]
add      add comma-separated chores
complete complete designated chore

Flags:
`, filepath.Base(os.Args[0]))
        flag.PrintDefaults()
    }
}

```

Listing 12-2: Initial housework application code (cmd/housework.go)

This bit of code sets up the command line arguments and their usage ❷: you can specify the `add` argument, followed by a comma-separated list of chores to add to the list, or you can pass the argument `complete` and a chore number to mark the chore as complete. In the absence of command line options, the app will display the current list of chores.

Since the ultimate purpose of this application is to demonstrate data serialization, you'll use multiple serialization formats to store the data. This should show you how easy it is to switch between various formats. To prepare for that, you include `import` statements for those formats ❶. This will make it easier for you to switch between the formats later.

Let's write the code to load chores from storage (see Listing 12-3).

```

--snip--

func load() ([]*housework.Chore, error) {
    if _, err := os.Stat(dataFile); ❶ os.IsNotExist(err) {
        return make([]*housework.Chore, 0), nil
    }

    df, err := ❷ os.Open(dataFile)
    if err != nil {
        return nil, err
    }
    defer func() {
        if err := df.Close(); err != nil {
            fmt.Printf("closing data file: %v", err)
        }
    }()

    return ❸ storage.Load(df)
}

```

Listing 12-3: Deserializing chores from a file (cmd/housework.go)

This function returns a slice of pointers to `housework.Chore` structs from Listing 12-1. If the data file does not exist ❶, you exit early, returning an empty slice. This default case will occur when you first run the application.

If the app finds a data file, you open it ❷ and pass it along to the storage's `Load` function ❸, which expects an `io.Reader`. You used the same pattern of accepting an interface and returning a concrete type in previous chapters.

Listing 12-4 defines a function that flushes the chores in memory to your storage for persistence.

```
--snip--

func flush(chores []*housework.Chore) error {
    df, err := ❶os.Create(dataFile)
    if err != nil {
        return err
    }
    defer func() {
        if err := df.Close(); err != nil {
            fmt.Printf("closing data file: %v", err)
        }
    }()

    return ❷storage.Flush(df, chores)
}
```

Listing 12-4: Flushing chores to the storage (cmd/housework.go)

Here, you create a new file or truncate the existing file ❶ and pass the file pointer and slice of chores to the storage's `Flush` function ❷. This function accepts an `io.Writer` and your slice. There's certainly room for improvement in the way you handle the existing serialized file. But for the purposes of demonstration, this will suffice.

You need a way to display the chores on the command line. Listing 12-5 adds such a function to your application.

```
--snip--

func list() error {
    chores, err := ❶load()
    if err != nil {
        return err
    }

    if len(chores) == 0 {
        fmt.Println("You're all caught up!")
        return nil
    }

    fmt.Println("#\t[X]\tDescription")
    for i, chore := range chores {
        c := " "
```

```

        if chore.Complete {
            c = "X"
        }
        fmt.Printf("%d\t[%s]\t%s\n", i+1, c, chore.Description)
    }

    return nil
}

```

Listing 12-5: Printing the list of chores to standard output (cmd/housework.go)

First, you load the list of chores from storage ❶. If there are no chores in your list, you simply print as much to standard output. Otherwise, you print a header and the list of chores, which looks something like this (see Listing 12-6).

#	[X]	Description
1	[]	Mop floors
2	[]	Clean dishes
3	[]	Mow the lawn

Listing 12-6: Example output of the list function with three chores in the list

The first column represents the chore number. You can reference this number to mark the chore complete, which will add an X between its brackets in the second column. The third column describes the chore.

Listing 12-7 implements the add function, which allows you to add chores to the list.

--snip--

```

func add(s string) error {
    chores, err := ❶load()
    if err != nil {
        return err
    }

    for _, chore := range ❷strings.Split(s, ",") {
        if desc := strings.TrimSpace(chore); desc != "" {
            chores = append(chores, &housework.Chore{
                Description: desc,
            })
        }
    }

    return ❸flush(chores)
}

```

Listing 12-7: Adding chores to the list (cmd/housework.go)

Unlike a long-running service, this application's lifetime starts when you execute it on the command line and ends when it completes whatever task you ask it to perform. Therefore, because you want your list of chores to persist between executions of the application, you need to store the chore state on disk. In other words, you retrieve the chores from storage ❶, modify them, and flush the changes to storage ❸. The changes persist until the next time you run the app.

You want the option to add more than one chore at a time, so you split the incoming chore description by commas ❷ and append each chore to the slice. Granted, this keeps you from using commas in individual chore descriptions, so the members of your household will have to keep their requests short (which isn't all bad, in my opinion). As an exercise, figure out a way around this limitation. One approach may be to use a different delimiter, but keep in mind, whatever you choose as a delimiter may have significance on the command line. Another approach may be to add support for quoted strings containing commas.

The last piece of this puzzle is my favorite part about working on chores: marking them as complete (see Listing 12-8).

--snip--

```
func complete(s string) error {
    i, err := strconv.Atoi(s)
    if err != nil {
        return err
    }

    chores, err := load()
    if err != nil {
        return err
    }

    if i < 1 || i > len(chores) {
        return fmt.Errorf("chore %d not found", i)
    }

    ❶ chores[i-1].Complete = true

    return flush(chores)
}
```

Listing 12-8: Marking a chore as complete (cmd/housework.go)

The complete function accepts the command line argument representing the chore you want to complete and converts it to an integer. I find I'm more efficient if I perform tasks one by one, so I'll have you mark only one complete at a time. You then load the chores from storage and make sure the integer is within range. If so, you mark the chore complete. Since you're

numbering chores starting with 1 when displaying the list, you need to account for placement in the slice by subtracting 1 ❶. Finally, you flush the chores to storage.

Now, let's tie everything together by implementing the app's main function in Listing 12-9.

```
--snip--

func main() {
    flag.Parse()

    var err error

    switch strings.ToLower(flag.Arg(0)) {
    case "add":
        err = add(strings.Join(flag.Args()[1:], " "))
    case "complete":
        err = complete(flag.Arg(1))
    }

    if err != nil {
        log.Fatal(err)
    }

    err = list()
    if err != nil {
        log.Fatal(err)
    }
}
```

Listing 12-9: The main logic of the housework application (cmd/housework.go)

You've put as much logic in the previous functions as possible, so this main function is quite minimal. You check the first argument to determine whether it's an expected subcommand. If so, you call the appropriate function. You call the list function if err is still nil after accounting for the optional subcommand and its arguments.

All that's left to do now is implement the storage Load and Flush functions for JSON, Gob, and protocol buffers.

JSON

JSON is a common, human-readable, text-based data serialization format that uses key-value pairs and arrays to represent complex data structures. Most contemporary languages offer official library support for JSON, which is one reason it's the customary encoding format for RESTful APIs.

JSON's types include strings, Booleans, numbers, arrays, key-value objects, and nil values specified by the keyword *null*. JSON numbers do not differentiate between floating-points and integers. You can read more about Go's JSON implementation at <https://blog.golang.org/json>.

Let's look at what the contents of the *housework.db* file would be if we JSON-encoded the chores from Listing 12-6. I've formatted the JSON for easier reading in Listing 12-10, though you could use the encoder's `SetIndent` method to do it for you.

```
❶ [
❷ {
    "Complete": false,
    "Description": "Mop floors"
  },
  {
    "Complete": false,
    "Description": "Clean dishes"
  },
  {
    "Complete": false,
    "Description": "Mow the lawn"
  }
]
```

*Listing 12-10: Formatted contents of the *housework.db* file after serializing the chores to JSON*

As you can see, the JSON is an array of objects **❶**, and each object **❷** includes `Complete` and `Description` fields and corresponding values.

Listing 12-11 details the JSON storage implementation using Go's `encoding/json` package.

```
package json

import (
    "encoding/json"
    "io"

    "github.com/awoodbeck/gnp/ch12/housework"
)

func Load(r io.Reader) ([]*housework.Chore, error) {
    var chores []*housework.Chore

    return chores, ❶ json.NewDecoder(r).Decode(&chores)
}

func Flush(w io.Writer, chores []*housework.Chore) error {
    return ❷ json.NewEncoder(w).Encode(chores)
}
```

Listing 12-11: JSON storage implementation (`json/housework.go`)

The `Load` function passes the `io.Reader` to the `json.NewDecoder` function **❶** and returns a decoder. You then call the decoder's `Decode` method, passing it a pointer to the `chores` slice. The decoder reads JSON from the `io.Reader`, deserializes it, and populates the `chores` slice.

The `Flush` function accepts an `io.Writer` and a `chores` slice. It then passes the `io.Writer` to the `json.NewEncoder` function ❷, which returns an encoder. You pass the `chores` slice to the encoder's `Encode` function, which serializes the `chores` slice into JSON and writes it to the `io.Writer`.

Now that you've implemented a JSON package that can serve as storage for your application, let's try it out in Listing 12-12.

```
$ go run cmd/housework.go
You're all caught up!
$ go run cmd/housework.go add Mop floors, Clean dishes, Mow the lawn
#      [X]      Description
1      [ ]      Mop floors
2      [ ]      Clean dishes
3      [ ]      Mow the lawn
$ go run cmd/housework.go complete 2
#      [X]      Description
1      [ ]      Mop floors
2      [X]      Clean dishes
3      [ ]      Mow the lawn
$ cat housework.db
[{"Complete":false,"Description":"Mop floors"},
{"Complete":true,"Description":"Clean dishes"},
{"Complete":false,"Description":"Mow the lawn"}]
```

Listing 12-12: Testing the housework application with JSON storage on the command line

Your first execution of the app lets you know you have nothing in your list of chores. You then add three comma-separated chores and complete the second one. Looks good. Notice also that the `housework.db` file contains readable JSON (to see this on Windows, use the `type` command instead of `cat`). Let's modify this application to use a binary encoding format native to Go.

Gob

Gob, as in “gobs of binary data,” is a binary serialization format native to Go. Engineers on the Go team developed Gob to combine the efficiency of protocol buffers, arguably the most popular binary serialization format, with JSON's ease of use. For example, protocol buffers don't let us simply instantiate a new encoder and throw data structures at it, as you did in the JSON example in Listing 12-11. On the other hand, Gob functions much the same way as the JSON encoder, in that Gob can intelligently infer an object's structure and serialize it.

If you're interested in exploring the motivation and finer points of Gob, give Rob Pike's “Gobs of Data” blog post a read (<https://blog.golang.org/gob>). In the meantime, let's implement our storage backend in Gob (see Listing 12-13).

```
package gob

import (
    "encoding/gob"
    "io"
```

```

    "github.com/awoodbeck/gnp/ch12/housework"
)

func Load(r io.Reader) ([]*housework.Chore, error) {
    var chores []*housework.Chore

    return chores, gob.NewDecoder(r).Decode(&chores)
}

func Flush(w io.Writer, chores []*housework.Chore) error {
    return gob.NewEncoder(w).Encode(chores)
}

```

Listing 12-13: Gob storage implementation (gob/housework.go)

If you're looking at this code and observing that it replaces all occurrences of json from Listing 12-11 with gob, you're not wrong. In Go, Gob is as easy to use as JSON, since it infers what it needs to encode from the object itself. You'll see how this differs from protocol buffers in the next section.

All that's left to do is swap out the JSON storage implementation for the Gob one by modifying the imports from Listing 12-2 (Listing 12-14).

```

--snip--
    "github.com/awoodbeck/gnp/ch12/housework"
❶ // storage "github.com/awoodbeck/gnp/ch12/json"
❷ storage "github.com/awoodbeck/gnp/ch12/gob"
    // storage "github.com/awoodbeck/gnp/ch12/protobuf"
--snip--

```

Listing 12-14: Swapping the JSON storage package for the Gob storage package (cmd/housework.go)

Comment out the JSON storage package import ❶ in Listing 12-2 and uncomment the Gob storage package one ❷.

Since your current *housework.db* file contains JSON, it isn't compatible with Gob. Therefore, the housework application will throw an error when attempting to decode it using Gob. Remove the *housework.db* file and retest the application (see Listing 12-15).

```

$ rm housework.db
$ go run cmd/housework.go
You're all caught up!
$ go run cmd/housework.go add Mop floors, Clean dishes, Mow the lawn
#      [X]      Description
1      [ ]      Mop floors
2      [ ]      Clean dishes
3      [ ]      Mow the lawn
$ go run cmd/housework.go complete 2
#      [X]      Description
1      [ ]      Mop floors
2      [X]      Clean dishes
3      [ ]      Mow the lawn
$ hexdump -c housework.db
00000000 \r 377 203 002 001 002 377 204 \0 001 377 202 \0 \0 ) 377

```

```

0000010 201 003 001 002 377 202 \0 001 002 001 \b C o m p l
0000020 e t e 001 002 \0 001 \v D e s c r i p t
0000030 i o n 001 \f \0 \0 \0 1 377 204 \0 003 002 \n M
0000040 o p f l o o r s \0 001 001 001 \f C l
0000050 e a n d i s h e s \0 002 \f M o w
0000060 t h e l a w n \0
000006a

```

Listing 12-15: Testing the housework application with Gob storage on the command line

Everything still works as expected. Using the hexdump tool, you can see that the *housework.db* file now includes binary data. It's certainly not human-readable as JSON was in Listing 12-12, but Go happily deserializes the Gob-encoded data, even though it's harder for us to make sense of it. (My Windows friends can find a hexdump binary at <https://www.di-mgt.com.au/hexdump-for-windows.html>, though you'll have to use the `-C` flag to get the same effect.)

If you are communicating with other Go services that support Gob, I recommend you use Gob over JSON. Go's `encoding/gob` is more performant than `encoding/json`. Gob encoding is also more succinct, in that Gob uses less data to represent objects than JSON does. This can make a difference when storing or transmitting serialized data.

Now that you have a taste for serializing data using `encoding/json` and `encoding/gob`, let's add protocol buffer support to your storage backend.

Protocol Buffers

Like Gob, *protocol buffers* use binary encoding to store or exchange information across various platforms. It's faster and more succinct than Go's JSON encoding. Unlike Gob and like JSON, protocol buffers are language neutral and enjoy wide support among popular programming languages. This makes them ideally suited for using in Go-based services that you hope to integrate with services written in other programming languages. This chapter assumes you're using the *proto3* version of the format.

Protocol buffers use a definition file, conventionally suffixed with *.proto*, to define messages. *Messages* describe the structured data you want to serialize for storage or transmission. For example, a protocol buffer message representing the Chore type looks like the definition in Listing 12-16.

```

message Chore {
    bool complete = 1;
    string description = 2;
}

```

Listing 12-16: Protocol buffer message definition representing a Chore type

You define a new message by using the `message` keyword, followed by the unique name of the message. Convention dictates you use Pascal case. (*Pascal casing* is a style of code formatting in which you concatenate capitalized words: *ThisIsPascalCasing*.) You then add fields to the Chore message. Each field definition includes a type, a snake-cased name, and a field number unique

to the message. (*Snake casing* is like Pascal casing except the first word is lowercase: *thisIsSnakeCasing*.) The field's type and number identify the field in the binary payload, so these must not change once used or you'll break backward compatibility. However, it's fine to add new messages and message fields to an existing *.proto* file.

Speaking of backward compatibility, it's a good practice to treat your protocol buffer definitions as you would an API. If third parties use your protocol buffer definition to communicate with your service, consider versioning your definition; this allows you to create new versions anytime you need to break backward compatibility. Your development can move forward with the latest version while clients continue to use the previous version until they're ready to target the latest version. You'll see one method of versioning the protocol buffer definitions later in this section.

You'll have to compile the *.proto* file to generate Go code from it. This code allows you to serialize and deserialize the messages defined in the *.proto* file. Third parties that want to exchange messages with you can use the same *.proto* file to generate code for their target programming language. The resulting code can exchange messages with you too. Therefore, before you can start using protocol buffers in earnest, you must install the protocol buffer compiler and its Go generation module. Your operating system's package manager may allow you to easily install the protocol buffer compiler. For example, on Debian 10, run the following:

```
$ sudo apt install protobuf-compiler
```

On macOS with Homebrew, run this:

```
$ brew install protobuf
```

On Windows, download the latest protocol buffer compiler ZIP file from <https://github.com/protocolbuffers/protobuf/releases/>, extract it, and add its *bin* subdirectory to your PATH. You should now be able to run the *protoc* binary on your command line.

A simple `go get` will install the protocol buffer's Go generator on your system. Make sure you have the resulting *protoc-gen-go* binary in your PATH or *protoc* won't recognize the plug-in:

```
$ GO111MODULE=on go get -u github.com/golang/protobuf/protoc-gen-go
```

Now that you've installed the protocol buffer compiler and Go generation module, let's create a new *.proto* file for your housework application (see Listing 12-17). You'll create this file in *housework/v1/housework.proto*. The *v1* in the path stands for *version 1* and allows you to introduce future versions of this package.

-
- ❶ `syntax = "proto3";`
 - ❷ `package housework;`
 - ❸ `option go_package = "github.com/awoodbeck/gnp/ch12/housework/v1/housework";`

```

message Chore {
    bool complete = 1;
    string description = 2;
}

message Chores {
    ❷repeated Chore chores = 1;
}

```

Listing 12-17: Protocol buffer definition for your housework application (housework/v1/housework.proto)

First, you specify that you’re using the proto3 syntax ❶ and that you want any generated code to have the package name `housework` ❷. Next, you add a `go_package` option ❸ with the full import path of the generated module. Then you define the `Chore` message and a second message named `Chores` that represents a collection of `Chore` messages based on the repeated field type ❹.

Now, let’s compile the `.proto` file to Go code:

```
$ protoc ❶--go_out=. ❷--go_opt=paths=source_relative housework/v1/housework.proto
```

You call `protoc` with flags indicating you want to generate Go code ❶ from the `housework/v1/housework.proto` file you created in Listing 12-17 and output the generated code to the `.proto` file’s current directory, with relative paths ❷.

If you receive the following error indicating `protoc` cannot find the `protoc-gen-go` binary, make sure `protoc-gen-go`’s location (likely `$GOPATH/bin`) is in your `PATH` environment variable:

```

protoc-gen-go: program not found or is not executable
--go_out: protoc-gen-go: Plugin failed with status code 1.

```

If `protoc` is happy with the `.proto` file and successfully generated the Go code, you’ll find that a new file named `housework/v1/housework.pb.go` exists with these first few lines, though the version numbers may differ. I’ll use the `head` command on Linux/macOS to print the first seven lines:

```

$ head -n 7 housework/v1/housework.pb.go
// Code generated by protoc-gen-go. DO NOT EDIT.
// versions:
//     protoc-gen-go v1.25.0
//     protoc        v3.6.1
// source: housework/v1/housework.proto

package housework

```

As the comments indicate, you shouldn’t edit this module. Instead, make any necessary changes to the `.proto` file and recompile it.

Now that you've generated a Go module from the *.proto* file, you can put it to effective use in Listing 12-18 by implementing your storage backend with protocol buffers.

```
package protobuf

import (
    "io"
    "io/ioutil"

    "google.golang.org/protobuf/proto"

    ❶ "github.com/awoodbeck/gnp/ch12/housework/v1"
)

func Load(r io.Reader) ([]*housework.Chore, error) {
    b, err := ioutil.ReadAll(r)
    if err != nil {
        return nil, err
    }

    var chores housework.Chores

    return chores.Chores, proto.Unmarshal(b, &chores)
}

func Flush(w io.Writer, chores []*housework.Chore) error {
    b, err := proto.Marshal(❷&housework.Chores{Chores: chores})
    if err != nil {
        return err
    }

    _, err = w.Write(b)

    return err
}
```

Listing 12-18: Protocol buffers storage implementation (protobuf/housework.go)

Instead of relying on the *housework* package from Listing 12-1, as you did when working with JSON and Gob, you import version 1 of the *protocol*-generated package, which you also named *housework* ❶. The generated *Chores* type ❷ is a struct with a *Chores* field, which itself is a slice of *Chore* pointers. Also, Go's protocol buffers package doesn't implement encoders and decoders. Therefore, you must marshal objects to bytes, write them to the *io.Writer*, and unmarshal bytes from the *io.Reader* yourself.

Revisit the code in Listing 12-2 and plug in the protocol buffers implementation with the two simple changes shown in Listing 12-19.

```
--snip--
❶ "github.com/awoodbeck/gnp/ch12/housework/v1"
// storage "github.com/awoodbeck/gnp/ch12/json"
```

```
// storage "github.com/awoodbeck/gnp/ch12/gob"
❷ storage "github.com/awoodbeck/gnp/ch12/protobuf"
--snip--
```

Listing 12-19: Swapping the JSON storage package for the protobuf storage package (cmd/housework.go)

You replace the housework package from Listing 12-1 with your generated package ❶, make sure to comment out the json and gob imports, and uncomment the protobuf storage import ❷. The actual functionality of the housework application remains unchanged.

Transmitting Serialized Objects

Although you may sometimes need to serialize and store objects locally, you're more likely to build a network service that serializes data. For example, an online store may have a web service that communicates with inventory, user accounting, billing, shipping, and notification services to facilitate customer orders. If these services ran on the same server, you'd have to buy a larger server to scale the online store as business grows. Another approach would be to run each service on its own server and increase the number of servers. But then you'd have a new problem: how can you share data among services when they no longer reside on a single server and so can't access the same memory?

Large technology companies facilitate this with *remote procedure calls (RPCs)*, a technique by which a client can transparently call a subroutine on a server as if the call were local. From your application's perspective, RPC services take code that appears to run locally and distribute it over a network. Your code may call a function that transparently relays a message to a server. The server would locally execute the function, then respond with the results, which your code receives as the function's return value. As far as your code is concerned, the function call is local, despite RPC's transparent interaction with the server. This approach allows you to scale services across servers while abstracting the details away from your code. In other words, your code functions the same no matter whether the function call runs on the same computer or on one over the network.

Most companies now implement RPC with *gRPC*, a cross-platform framework that leverages HTTP/2 and protocol buffers. Let's use it here to build something more sophisticated than an app to keep track of the housework you have yet to do. You'll write a service that can send tasks to Rosie, the robotic maid from the classic animated series *The Jetsons*, who can take over your domestic responsibilities. Granted, she won't be available until the year 2062, but you can get a head start on the code.

Connecting Services with gRPC

The gRPC framework is a collection of libraries that abstracts many of the RPC implementation details. It is platform neutral and programming-language agnostic; you can use it to integrate a Go service running on

Windows with a Rust service running on Linux, for example. Now that you know how to work with protocol buffers, you have the foundation needed to add gRPC support to your application. You'll reuse the *.proto* file from the previous section.

First, make sure your gRPC package is up-to-date:

```
$ go get -u google.golang.org/grpc
```

Next, get the appropriate module for generating gRPC Go code:

```
$ go get -u google.golang.org/grpc/cmd/protoc-gen-go-grpc
```

The protocol buffer compiler includes a gRPC module. This module will output Go code that lets you easily add gRPC support. First, you need to add definitions to the *.proto* file. Listing 12-20 defines a new service and two additional messages.

```
--snip--

service RobotMaid {
❶  rpc Add (Chores) returns (Response);
    rpc Complete (CompleteRequest) returns (Response);
    rpc List (Empty) returns (Chores);
}

message CompleteRequest {
    int32 ❷ chore_number = 1;
}

❸ message Empty {}

message Response {
    string message = 1;
}
```

Listing 12-20: Additional definitions to support a gRPC RobotMaid service (housework/v1/housework.proto)

The service needs to support the same three calls you used on the command line: add, complete, and list. You define a new service named *RobotMaid*, then add three RPC methods to it. These RPC methods correspond to the functions defined in Listings 12-5, 12-7, and 12-8 for use on the command line: the list, add, and complete functions, respectively. Instead of calling these functions locally, you'll call the corresponding method on the *RobotMaid* to execute these commands via RPC. You prefix each method with the *rpc* keyword and follow this with the Pascal-cased method name. Next, you write the request message type in parentheses, the *returns* keyword, and the return message type in parentheses ❶.

The List method doesn't require any user input, but as in the command line application, you still must provide a request message type for it, even if it's nil. In gRPC, the message type equivalent to nil is an empty message, which you call *Empty* ❸.

Until you've had the opportunity to add proper artificial intelligence (AI) to the robot, you'll need to tell Rosie when her current chore is complete so she can move on to the next one. For this, you add a new message that informs her of the completed chore number ❷. Since you expect feedback from Rosie, you also add a response message that contains a string.

Now compile the *.proto* file to use the new service and messages. Tell *protoc* to use the *protoc-gen-go-grpc* binary, which must also be in your *PATH* environment variable, like this:

```
$ protoc ❶--go-grpc_out=. ❷--go-grpc_opt=paths=source_relative \
housework/v1/housework.proto
```

The `--go-grpc_out` flag ❶ invokes the *protoc-gen-go-grpc* binary to add gRPC support to the generated code. This binary generates the relevant gRPC service code for you and writes gRPC-specific code to the *housework/v1/housework_grpc.pb.go* file since you told *protoc-gen-go-grpc* to use relative paths ❷. You can now use the generated code to build a gRPC server and client.

Creating a TLS-Enabled gRPC Server

Now let's implement a gRPC client and server. By default, gRPC requires a secure connection, so you'll add TLS support to your server. You'll use the server's *cert.pem* and *key.pem* files created in the preceding chapter for your gRPC server and pin the server's certificate to your client. See the "Generating Certificates for Authentication" section on page 256 for details.

You'll leverage the Go code generated by your *.proto* file to define a new *RobotMaid* client and server and use the client to communicate with the server over the network using gRPC. First, let's create the server portion of your robot maid. The *RobotMaidServer* interface generated from your *.proto* file looks like this:

```
type RobotMaidServer interface {
    Add(context.Context, *Chores) (*Response, error)
    Complete(context.Context, *CompleteRequest) (*Response, error)
    List(context.Context, *empty.Empty) (*Chores, error)
    mustEmbedUnimplementedRobotMaidServer()
}
```

You'll implement this interface in Listing 12-21 by creating a new type named *Rosie*.

```
package main

import (
    "context"
    "fmt"
    "sync"

    "github.com/awoodbeck/gnp/ch12/housework/v1"
)
```

```

type Rosie struct {
    mu sync.Mutex
    ❶ chores []*housework.Chore
}

func (r *Rosie) Add(_ context.Context, chores *housework.Chores) (
    *housework.Response, error) {
    r.mu.Lock()
    r.chores = append(r.chores, chores.Chores...)
    r.mu.Unlock()

    return ❷&housework.Response{Message: "ok"}, nil
}

func (r *Rosie) Complete(_ context.Context,
    req *housework.CompleteRequest) (*housework.Response, error) {
    r.mu.Lock()
    defer r.mu.Unlock()

    if r.chores == nil || req.ChoreNumber < 1 ||
        int(req.ChoreNumber) > len(r.chores) {
        return nil, fmt.Errorf("chore %d not found", req.ChoreNumber)
    }

    r.chores[req.ChoreNumber-1].Complete = true

    return &housework.Response{Message: "ok"}, nil
}

func (r *Rosie) List(_ context.Context, _ *housework.Empty) (
    *housework.Chores, error) {
    r.mu.Lock()
    defer r.mu.Unlock()

    if r.chores == nil {
        r.chores = make([]*housework.Chore, 0)
    }

    return &housework.Chores{Chores: r.chores}, nil
}

func (r *Rosie) Service() *housework.RobotMaidService {
    return ❸&housework.RobotMaidService{
        Add:      r.Add,
        Complete: r.Complete,
        List:     r.List,
    }
}

```

Listing 12-21: Building a RobotMaid-compatible type named Rosie (server/rosie.go)

The new Rosie struct keeps its list of chores in memory ❶, guarded by a mutex, since more than one client can concurrently use the service. The Add, Complete, and List methods all return either a response message type ❷ or an error, both of which ultimately make their way back to the client.

The `Service` method returns a pointer to a new `housework.RobotMaidService` instance ❸ where `Rosie`'s `Add`, `Complete`, and `List` methods map to their corresponding method on the new instance.

Now, let's set up a new gRPC server instance by using the `Rosie` struct (Listing 12-22).

```
package main

import (
    "crypto/tls"
    "flag"
    "fmt"
    "log"
    "net"

    "google.golang.org/grpc"

    "github.com/awoodbeck/gnp/ch12/housework/v1"
)

var addr, certFn, keyFn string

func init() {
    flag.StringVar(&addr, "address", "localhost:34443", "listen address")
    flag.StringVar(&certFn, "cert", "cert.pem", "certificate file")
    flag.StringVar(&keyFn, "key", "key.pem", "private key file")
}

func main() {
    flag.Parse()

    server := ❶ grpc.NewServer()
    rosie := new(Rosie)
    ❷ housework.RegisterRobotMaidServer(server, ❸ rosie.Service())

    cert, err := tls.LoadX509KeyPair(certFn, keyFn)
    if err != nil {
        log.Fatal(err)
    }

    listener, err := net.Listen("tcp", addr)
    if err != nil {
        log.Fatal(err)
    }

    fmt.Printf("Listening for TLS connections on %s ...", addr)
    log.Fatal(
        ❹ server.Serve(
            ❺ tls.NewListener(
                listener,
                &tls.Config{
                    Certificates:      [][]tls.Certificate{cert},
                    CurvePreferences:  []tls.CurveID{tls.CurveP256},
                    MinVersion:         tls.VersionTLS12,
                },
            ),
        ),
    )
}
```

```

        PreferServerCipherSuites: true,
    },
),
)
}

```

Listing 12-22: Creating a new gRPC server using Rosie (server/server.go)

First, you retrieve a new server instance ❶. You pass it and a new `*housework.RobotMaidService` from Rosie's `Service` ❸ method to the `RegisterRobotMaidServer` function ❷ in the generated gRPC code. This registers Rosie's `RobotMaidService` implementation with the gRPC server. You must do this before you call the server's `Serve` method ❹. You then load the server's key pair and create a new TLS listener ❺, which you pass to the server when calling `Serve`.

Now that you have a gRPC server implementation, let's work on the client.

Creating a gRPC Client to Test the Server

The client-side code isn't much different from what you wrote in the "Serializing Objects" section on page 270. The main difference is that you need to instantiate a new gRPC client and modify the `add`, `complete`, and `list` functions to use it. Remember, you can implement the client portion in a separate programming language if the programming language offers protobuf support. You can generate code for your target language from your `.proto` file with the expectation it will work seamlessly with your server in Listing 12-22.

Listing 12-23 details the changes to Listing 12-2 necessary to add gRPC support to the housework application.

```

package main

import (
    ❶ "context"
    ❷ "crypto/tls"
    ❸ "crypto/x509"
    "flag"
    "fmt"
    ❹ "io/ioutil"
    "log"
    "os"
    "path/filepath"
    "strconv"
    "strings"

    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials"

    "github.com/awoodbeck/gnp/ch12/housework/v1"
)

```

```

var addr, caCertFn string

func init() {
    ❶ flag.StringVar(&addr, "address", "localhost:34443", "server address")
    ❷ flag.StringVar(&caCertFn, "ca-cert", "cert.pem", "CA certificate")

    flag.Usage = func() {
        fmt.Fprintf(flag.CommandLine.Output(),
            `Usage: %s [flags] [add chore, ...|complete #]
        add      add comma-separated chores
        complete  complete designated chore
    }

Flags:
`, filepath.Base(os.Args[0]))
    flag.PrintDefaults()
}

```

Listing 12-23: Initial gRPC client code for our housework application (client/client.go)

Aside from all the new imports ❶ ❷ ❸ ❹, you add flags for the gRPC server address ❺ and its certificate ❻.

Listing 12-24 uses the gRPC client to list the current chores.

```

--snip--

func list(ctx context.Context, client housework.RobotMaidClient) error {
    chores, err := client.List(ctx, ❶new(housework.Empty))
    if err != nil {
        return err
    }

    if len(chores.Chores) == 0 {
        fmt.Println("You have nothing to do!")
        return nil
    }

    fmt.Println("#\t[X]\tDescription")
    for i, chore := range chores.Chores {
        c := " "
        if chore.Complete {
            c = "X"
        }
        fmt.Printf("%d\t[%s]\t%s\n", i+1, c, chore.Description)
    }

    return nil
}

```

Listing 12-24: Using the gRPC client to list the current chores (client/client.go)

This code is quite like Listing 12-5, except you're asking the gRPC client for the list of chores, which retrieves them from the server. You need to pass along an empty message to make gRPC happy ❶.

Listing 12-25 uses the gRPC client to add new chores to the gRPC server's chores.

--snip--

```
func add(ctx context.Context, client housework.RobotMaidClient,
s string) error {
    chores := new(housework.Chores)

    for _, chore := range strings.Split(s, ",") {
        if desc := strings.TrimSpace(chore); desc != "" {
            chores.Chores = append(chores.Chores, &housework.Chore{
                Description: desc,
            })
        }
    }

    var err error
    if len(chores.Chores) > 0 {
        _, err = client.Add(ctx, chores)
    }

    return err
}
```

Listing 12-25: Adding new chores using the gRPC client (client/client.go)

As you did in the previous section, you parse the comma-separated list of chores. Instead of flushing these chores to disk, you pass them along to the gRPC client. The gRPC client transparently sends them to the gRPC server and returns the response to you. Since you know Rosie returns a non-nil error when the Add call fails, you return the error ❶ as the result of the add function.

In Listing 12-26, you write the code to mark chores complete. Doing this over gRPC requires a bit less code than Listing 12-8 since most of the logic is in the server.

--snip--

```
func complete(ctx context.Context, client housework.RobotMaidClient,
s string) error {
    i, err := strconv.Atoi(s)
    if err == nil {
        _, err = client.Complete(ctx,
            &housework.CompleteRequest{❶ChoreNumber: int32(i)})
    }

    return err
}
```

Listing 12-26: Marking chores complete by using the gRPC client (client/client.go)

Notice the *protoc-gen-go* module, which converts the snake-cased `chore_number` field in Listing 12-20 to Pascal case in the generated Go code ❶. You must also convert the `int` returned by `strconv.Atoi` to an `int32` before assigning it to the complete request message's chore number since `ChoreNumber` is an `int32`.

Listing 12-27 creates a new gRPC connection and pins the server's certificate to its TLS configuration.

--snip--

```
func main() {
    flag.Parse()

    caCert, err := ioutil.ReadFile(caCertFn)
    if err != nil {
        log.Fatal(err)
    }
    certPool := x509.NewCertPool()
    if ok := certPool.AppendCertsFromPEM(caCert); !ok {
        log.Fatal("failed to add certificate to pool")
    }

    conn, err := ❶ grpc.Dial(
        addr,
        ❷ grpc.WithTransportCredentials(
            ❸ credentials.NewTLS(
                &tls.Config{
                    CurvePreferences: []tls.CurveID{tls.CurveP256},
                    MinVersion:         tls.VersionTLS12,
                    RootCAs:            certPool,
                },
            ),
        ),
    )
    if err != nil {
        log.Fatal(err)
    }
}
```

Listing 12-27: Creating a new gRPC connection using TLS and certificate pinning (client/client.go)

On the client side, you first create a new gRPC network connection ❶ and then use the network connection to instantiate a new gRPC client. For most use cases, you can simply pass the address to `grpc.Dial`. But you want to pin the server's certificate to the client connection. Therefore, you need to explicitly pass in a `grpc.DialOption` with the appropriate TLS credentials. This involves using the `grpc.WithTransportCredentials` function ❷ to return the `grpc.DialOption` and the `credentials.NewTLS` function ❸ to create the transport credentials from your TLS configuration. The result is a gRPC network connection that speaks TLS with the server and authenticates the server by using the pinned certificate.

You use this gRPC network connection to instantiate a new gRPC client in Listing 12-28.

--snip--

```
rosie := ❶housework.NewRobotMaidClient(conn)
ctx := context.Background()

switch strings.ToLower(flag.Arg(0)) {
case "add":
    err = add(ctx, rosie, strings.Join(flag.Args()[1:], " "))
case "complete":
    err = complete(ctx, rosie, flag.Arg(1))
}

if err != nil {
    log.Fatal(err)
}

err = list(ctx, rosie)
if err != nil {
    log.Fatal(err)
}
}
```

Listing 12-28: Instantiating a new gRPC client and making calls (client/client.go)

Aside from instantiating a new gRPC client from the gRPC network connection ❶, this bit of code doesn't vary much from Listing 12-9. The difference, of course, lies in the fact that any interaction with the list of chores transparently takes place over a TLS connection to the gRPC server.

Give it a try. In one terminal, start the server:

```
$ go run server/server.go server/rosie.go -cert server/cert.pem -key server/
key.pem
Listening for TLS connections on localhost:34443 ...
```

And in another terminal, run the client:

```
$ go run client/client.go -ca-cert server/cert.pem
You have nothing to do!
$ go run client/client.go -ca-cert server/cert.pem add Mop floors, Wash dishes
#      [X]      Description
1      [ ]      Mop floors
2      [ ]      Wash dishes
$ go run client/client.go -ca-cert server/cert.pem complete 2
#      [X]      Description
1      [ ]      Mop floors
2      [X]      Wash dishes
```

Of course, restarting the server wipes out the list of chores. I'll leave it as an exercise for you to implement persistence on the server. One approach is to have the server load the chores from and flush chores to disk as you did earlier in this chapter.

What You've Learned

Data serialization allows you to exchange data in a platform-neutral and language-neutral way. You can also serialize data for long-term storage, retrieve and deserialize the data, and pick up where your application left off.

JSON is arguably the most popular text-based data serialization format. Contemporary programming languages offer good support for JSON, which is one reason for its ubiquity in RESTful APIs. Go offers good support for binary-based data serialization formats as well, including Gob, which is nearly a drop-in replacement for JSON. Gob is Go's native binary data serialization format, and it's designed to be efficient and easy to use.

If you're looking for a binary data serialization format with wider support, consider protocol buffers. Google designed protocol buffers to facilitate the exchange of serialized binary data across its supported platforms and programming languages. Many contemporary programming languages currently offer support for protocol buffers. Although protocol buffers aren't the same drop-in replacement in Go as Gob is for JSON, Go has excellent protocol buffer support, nonetheless. You first need to add definitions that define the data structures you intend to serialize in a *.proto* file. You then use the protocol buffer compiler and its Go module to generate Go code that corresponds to your defined data structures. Finally, you use the generated code to serialize your data structures into protocol buffer-formatted binary data and deserialize that binary data back into your data structures.

The gRPC framework is a high-performance, platform-neutral standard for making distributed function calls across a network. The *RPC* in *gRPC* stands for *remote procedure call*, which is a technique for transparently calling a function on a remote system and receiving the result as if you had executed the function on your local system. gRPC uses protocol buffers as its underlying data serialization format. Go's protocol buffer module allows you to easily add gRPC support by defining services in your *.proto* file and leveraging the generated code. This lets you quickly and efficiently stand up distributed services or integrate with existing gRPC services.

13

LOGGING AND METRICS



In an ideal world, our code would be free of bugs from the outset. Our network services would exceed our expectations for performance and capacity, and they would be robust enough to adapt to unexpected input without our intervention. But in the real world, we need to worry about unexpected and potentially malicious input, hardware degradation, network outages, and outright bugs in our code.

Monitoring our applications, no matter whether they are on premises or in the cloud, is vital to providing resilient, functional services to our users. Comprehensive logging allows us to receive timely details about errors, anomalies, or other actionable events, and metrics give us insight into the current state of our services, as well as help us identify bottlenecks. Taken together, logging and metrics allow us to manage service issues and focus our development efforts to avoid future failures.

You’ve used Go’s `log` and `fmt` packages to give you feedback in previous chapters, but this chapter will take a deeper dive into logging and instrumenting your services. You will learn how to use log levels to increase or decrease the verbosity of your logs and when to use each log level. You’ll learn how to add structure to your log entries so software can help you make better sense of log entries and zero in on relevant logs. I’ll introduce you to the concept of wide event logging, which will help you maintain a handle on the amount of data you log as your services scale. You’ll learn techniques for dynamically enabling debug logging and managing log file rotation from your code.

This chapter will also introduce you to Go kit’s `metrics` package. Per Go kit’s documentation, the `metrics` package “provides a set of uniform interfaces for service instrumentation.” You’ll learn how to instrument your services by using counters, gauges, and histograms.

By the end of this chapter, you should have a handle on how to approach logging, how to manage log files to prevent them from consuming too much hard drive space, and how to instrument your services to gain insight into their current state.

Event Logging

Logging is hard. Even experienced developers struggle to get it right. It’s tough to anticipate what questions you’ll need your logs to answer in the future, when your service fails—yet you should resist the urge to log everything just in case. You need to strike a balance in order to log the right information to answer those questions without overwhelming yourself with irrelevant log lines. Overzealous logging may suit you fine in development, where you control the scale of testing and overall entropy of your service, but it will quickly degrade your ability to find the needle in the haystack when you need to diagnose production failures.

In addition to figuring out what to log, you need to consider that logging isn’t free. It consumes CPU and I/O time your application could otherwise use. A log entry added to a busy for loop while in development may help you understand what your service is doing. But it may become a bottleneck in production, insidiously adding latency to your service.

Instead, sampling these log entries, or logging on demand, may provide suitable compromises between log output and overhead. You might find it helpful to use *wide event* log entries, which summarize a transaction. For example, a service in development may log half a dozen entries about a request, any intermediate steps, and a response. In production, a single wide event log entry providing these details scales better. You’ll learn more about wide event log entries in “Scaling Up with Wide Event Logging” on page 312.

Lastly, logging is subjective. An anomaly may be inconsequential in my application but indicative of a larger issue in your application. Whereas I could ignore the anomaly, you’d likely want to know about it. For this

reason, it's best if we discuss logging in terms of best practices. These practices are a good baseline approach, but you should tailor them to each application.

The log Package

You have superficial experience using Go's log package, in earlier chapters, for basic logging needs, like timestamping log entries and optionally exiting your application with `log.Fatal`. But it has a few more features we have yet to explore. These require us to go beyond the package-level logger and instantiate our own `*log.Logger` instance. You can do this using the `log.New` function:

```
func New(out io.Writer, prefix string, flag int) *Logger
```

The `log.New` function accepts an `io.Writer`, a string prefix to use on each log line, and a set of flags that modify the logger's output. Accepting an `io.Writer` means the logger can write to anything that satisfies that interface, including an in-memory buffer or a network socket.

The default logger writes its output to `os.Stderr`, standard error. Let's look at an example logger in Listing 13-1 that writes to `os.Stdout`, standard output.

```
func Example_log() {  
    l := log.New(❶os.Stdout, ❷"example: ", ❸log.Lshortfile)  
    l.Print("logging to standard output")  
  
    // Output:  
    // example: ❹log_test.go:12: logging to standard output  
}
```

Listing 13-1: Writing a log entry to standard output (`log_test.go`)

You create a new `*log.Logger` instance that writes to standard output ❶. The logger prefixes each line with the string *example:* ❷. The flags of the default logger are `log.Ldate` and `log.Ltime`, collectively `log.LstdFlags`, which print the timestamp of each log entry. Since you want to simplify the output for testing purposes when you run the example on the command line, you omit the timestamp and configure the logger to write the source code filename and line of each log entry ❸. The `l.Print` function on line 12 of the `log_test.go` file results in the output of those values ❹. This behavior can help with development and debugging, allowing you to zero in on the exact file and line of an interesting log entry.

Recognizing that the logger accepts an `io.Writer`, you may realize this allows you to use multiple writers, such as a log file and standard output or an in-memory ring buffer and a centralized logging server over a network. Unfortunately, the `io.MultiWriter` is not ideal for use in logging. An `io.MultiWriter` writes to each of its writers in sequence, aborting if it receives an error from any `Write` call. This means that if you configure

your `io.MultiWriter` to write to a log file and standard output in that order, standard output will never receive the log entry if an error occurred when writing to the log file.

Fear not. It's an easy problem to solve. Let's create our own `io.MultiWriter` implementation, starting in Listing 13-2, that sustains writes across its writers and accumulates any errors it encounters.

```
package ch13

import (
    "io"

    "go.uber.org/multierr"
)

type sustainedMultiWriter struct {
    writers []io.Writer
}

func (s *sustainedMultiWriter) ❶Write(p []byte) (n int, err error) {
    for _, w := range s.writers {
        i, wErr := ❷w.Write(p)
        n += i
        err = ❸multierr.Append(err, wErr)
    }

    return n, err
}
```

Listing 13-2: A multiwriter that sustains writing even after receiving an error (writer.go)

As with `io.MultiWriter`, you'll use a struct that contains a slice of `io.Writer` instances for your sustained multiwriter. Your multiwriter implements the `io.Writer` interface ❶, so you can pass it into your logger. It calls each writer's `Write` method ❷, accumulating any errors with the help of Uber's `multierr` package ❸, before ultimately returning the total written bytes and cumulative error.

Listing 13-3 adds a function to initialize a new sustained multiwriter from one or more writers.

```
--snip--

func SustainedMultiWriter(writers ...io.Writer) io.Writer {
    mw := &sustainedMultiWriter{writers: ❶make([]io.Writer, 0, len(writers))}

    for _, w := range writers {
        if m, ok := ❷w.(*sustainedMultiWriter); ok {
            mw.writers = ❸append(mw.writers, m.writers...)
            continue
        }

        mw.writers = ❹append(mw.writers, w)
    }
}
```